

FOOFOO

Comprehensive Product Requirements Document & Intelligence Bibles

Version: 1.0 **Date:** 5 August 2026 **Status:** Product and engineering baseline for review **Audience:** Founders, Product, Design, Data Science, Food Science, Engineering, Growth, Operations **Scope:** India-first, globally extensible AI meal-decision platform

Product promise: Foofoo removes the recurring cognitive burden of “Aaj khaane mein kya banaye?” by producing a safe, culturally fluent, household-aware plan that people can trust, change, cook, or order.

This document is deliberately one integrated artifact. Part I is the complete PRD. Part II is the **FooFoo Recommendation Engine Bible**. Part III is the **Food Intelligence Bible**. Part IV is the **Engineering & Architecture Bible**, including the expected database schema, data provenance, APIs, MLOps, and the exact retrieval path used to recommend meals to one household.

Source rule. The active Product, Architecture, Research, Roadmap, Visual, and current-status documents in the repository were used. docs/archive/**, database/archive/**, docs/governance/**, and documents whose principal purpose is governance were excluded. Where an active design and live implementation differ, the difference is labelled **target**, **current**, or **decision required**.

Document map

Part	Purpose	Primary reader
I. Product PRD	Vision, market, users, psychology, journeys, UX, screens, growth, GTM, revenue, metrics	Product, Design, Growth
II. Recommendation Engine Bible	Candidate generation, scoring, ranking, cold start, household aggregation, learning, experimentation	Data Science, ML, Product
III. Food Intelligence Bible	Meal Genome, ontology, graph, nutrition, regional and temporal intelligence	Food Science, Data, ML
IV. Engineering & Architecture Bible	Platform, services, APIs, events, database, provenance, infrastructure, MLOps, security	Engineering, Data Platform, SRE
V. Delivery plan	v1→v5 roadmap, acceptance criteria, risks, decision log	Leadership, PMO

The system is inspired by three patterns, adapted rather than copied:

- **Netflix:** multi-stage candidate generation, contextual ranking, continuous experimentation, and explainable personalization.
- **Amazon:** a retail-grade canonical catalog, strict identifiers, event streams, operational services, and independently scalable domains.
- **Spotify:** taste vectors, a blend of familiar and exploratory items, session context, and a product loop that turns light feedback into durable personalization.

The essential difference is that a meal is not media. Recommendations must satisfy household-wide safety constraints, cooking feasibility, meal composition, regional identity, recent consumption, nutrition, ingredient availability, and the unequal preferences of adults, children, elders, and the primary cook.

Part I — Comprehensive Product Requirements Document

1. Executive summary

Foofoo creates a new consumer utility category: the **meal decision layer**. Delivery platforms answer how to obtain food; recipe platforms answer how to cook a selected dish; trackers describe what was already eaten. Foofoo answers what a household should eat next, then connects that decision to cooking, ordering, and learning.

The beachhead is an urban Indian household where one person carries most of the planning burden. The product opens to a pre-built day and week, not an empty search box. A household can lock a choice, swipe alternatives, mark “not today,” permanently exclude a dish, add a dish to another day, and record whether it was cooked or ordered. Every action becomes evidence; none may override allergies, religious rules, or explicit exclusions.

The launch wedge is convenience and emotional relief, not clinical nutrition. Nutrition supports balance and transparency but Foofoo is not a doctor. The moat comes from four compounding assets: a culturally precise meal ontology, a Meal Genome, household decision history, and a recommendation event corpus with decision traces.

North-star outcome: households complete more intended home-meal decisions with less planning effort.

North-star metric: Weekly Successful Meal Decisions (WSMD): distinct planned meal slots per household that are accepted or locked and subsequently marked cooked, ordered, or implicitly confirmed without a replacement.

MVP guardrails: zero known hard-constraint violations; p95 cached plan load under 1 second on a budget Android device; p95 fresh recommendation under 3 seconds; no child or health-condition claim presented as medical advice; deletion/export paths available; degraded mode returns a previously safe plan.

2. Problem definition and jobs to be done

Meal choice is a repeated constrained-optimization problem disguised as a simple question. An Indian household may decide breakfast, lunch, tiffin, snack, and dinner while balancing taste, availability, preparation time, cook skill, freshness, multiple diets, fasting, weather, leftovers, and the memory of recent meals. The decision maker typically optimizes this mentally with incomplete information and absorbs the social cost of a poor choice.

Functional jobs

1. “Build a realistic plan for my household before I have to think about it.”
2. “Give me alternatives that are meaningfully different but still feasible.”
3. “Remember what everyone cannot, will not, and recently did eat.”
4. “Help me convert a plan into cooking, ingredients, or an order.”
5. “Adapt when today changes without destroying the rest of my week.”

Emotional jobs

- Reduce decision fatigue, guilt, and the feeling that meal work is invisible.
- Preserve agency: a recommendation is a helpful default, never a command.
- Create confidence that “this works for us,” not merely “this is popular.”
- Share the planning load without forcing every member into another app.

Social jobs

- Avoid recurring “anything is fine” / post-decision complaint dynamics.
- Include children and elders without giving every preference equal veto power.
- Respect regional identity after migration and mixed-culture households.
- Make the planner feel competent rather than judged.

Non-goal: maximizing session time. Foofoo should often succeed in under two minutes.

3. Product vision and principles

Ten-year vision: a universal food-intelligence layer that can reason from household identity, health constraints, culture, context, inventory, budget, and intent to an actionable meal—then coordinate the surrounding ecosystem.

Product principles

Principle	Requirement
Decide first	The home screen leads with a recommended plan, not content discovery.
Household, not account	The unit of optimization is the eating group; the account is only an access boundary.
Safety before relevance	Hard constraints filter candidates before ranking and are rechecked after ranking.
Classes before dishes	Plan meal roles and composition before selecting specific dishes.
Familiar plus fresh	Use controlled exploration; novelty is a dose, not a goal.
Explain the useful why	Show “quick, familiar, good for a rainy weekday,” not opaque scores.
Corrections are gifts	“Never,” “not today,” swaps, locks, cooks, and orders are learning signals.
Regional identity is layered	Home state, current city, household tradition, season, and exposure are separate signals.
Low attention wins	Defaults must be good enough that silence can mean acceptance.
Earn expansion	Recipes, groceries, commerce, and clinical partnerships follow trust in the core decision.

Product boundaries

Foofoo may recommend and educate; it must not diagnose, prescribe treatment, guarantee health outcomes, infer sensitive health status without consent, or permit a commercial partner to buy rank position disguised as relevance.

4. Market research — India

India is structurally attractive because home cooking remains culturally central while urban work patterns increase time poverty. Regional food identities are unusually dense, meal composition commonly spans multiple dishes, vegetarian and religious constraints are prevalent, and low-cost mobile data makes daily utility feasible.

Current public indicators support distribution readiness. TRAI reported more than one billion broadband subscriptions by March 2026; NPCI reported more than 23 billion UPI transactions in May 2026. These are infrastructure signals, not proof of willingness to use or pay for a planner. WHO India notes that urbanization and lifestyle change are shifting dietary patterns while healthy diets still depend on individual need, locally available foods, and dietary customs.

India segmentation thesis

Segment	Core tension	Product wedge	Monetization likelihood
Dual-income families	Time + coordination	Ready-made household plan	Medium-high
Primary homemakers	Invisible mental load	Relief + variety + recognition	Medium
Solo professionals	Delivery default + waste	Quick single-serve plan	Medium
Fitness/health-aware adults	Indian relevance gap	Goal-aware planning	High, with safe scope
Joint families	Conflicting needs	Base meal + member add-ons	Medium-high
Students/shared flats	Budget + beginner skill	Cheap, easy, social plan	Low ARPU, high referrals
Migrant households	Identity + local exposure	Home/current-city blend	Medium

Repository market numbers such as TAM, SAM, and competitor revenue are treated as internal hypotheses because commercial research values can change and may use incompatible definitions. The board-ready sizing model should be bottoms-up: reachable households × activation × retained planning households × paid conversion × ARPU.

Sources: [TRAI dashboard](#), [NPCI UPI statistics](#), [WHO India healthy diet](#).

5. Market research — global

Globally, Foofoo sits between recipe discovery, grocery planning, calorie tracking, meal kits, food delivery, and general-purpose AI assistants. Every adjacency validates a piece of demand, but none necessarily owns the household decision graph.

Competitive archetypes

Archetype	Strength	Structural gap Foofoo targets
Recipe publishers and creator video	Inspiration and execution	Assume the user has chosen a dish; weak household memory
Calorie and diet trackers	Nutrient databases and goal tracking	Retrospective, individual, high-friction logging
Meal-planning utilities	Calendars, lists, recipe import	Rules and manual configuration more than adaptive intelligence
Meal kits	Closed-loop fulfillment	Limited geography, cost, cuisine, and household flexibility
Delivery marketplaces	Supply, logistics, transaction data	Optimize restaurant ordering, not home meal planning
Grocery retailers	Basket and purchase knowledge	Ingredient-centric; unclear meal intent and shared preferences
General AI assistants	Generative breadth	No durable food master, household safety model, or closed feedback loop

Global expansion sequence

1. Validate the household-decision loop in India.
2. Build locale packs that separate universal ontology from cultural policy.
3. Enter diaspora corridors where Indian identity and local ingredient availability intersect.
4. Extend to culturally complex markets with high home-cooking frequency.
5. Offer food-intelligence APIs to retailers, wellness platforms, and appliance ecosystems.

The global opportunity is not “more recipes.” It is decision infrastructure: representing eaters, meals, food, context, and outcomes in one governed learning system.

6. Positioning and category design

Category: AI meal decision assistant. **For:** households and individuals who repeatedly ask what to eat. **Foofoo:** builds a practical, personalized meal plan before the decision becomes stressful. **Unlike:** delivery, recipe, and tracking apps, Foofoo understands household composition, regional identity, recent meals, constraints, and context. **Proof:** a safe plan is ready on open, every correction improves the next one, and the user can trace why an item appeared.

Messaging ladder

Layer	Message
Functional	“Today’s meals are already planned.”
Emotional	“One less thing to carry.”
Household	“A plan that works for everyone at home.”
Intelligence	“It learns what your family actually eats.”
Cultural	“Indian meals, regions, seasons, and realities—not translated Western templates.”

Category traps to avoid

- “AI nutritionist” creates medical expectations and regulatory risk.
 - “Recipe app” demotes Foofoo to content search.
 - “Diet plan” feels restrictive and individual.
 - “Food social network” rewards attention rather than completed decisions.
 - “Grocery list app” describes an output, not the core value.
- The product should be branded around calm competence: warm, grounded, optimistic, and never scolding.

7. Persona system — why 25, not 25 separate algorithms

Personas are empathy and testing tools. The engine must not hard-code fixed people. It represents users through composable dimensions: household structure, life stage, diet, health overlays, region, migration, cook skill, time pressure, budget, and novelty tolerance.

Personas 1-9: core India households

#	Persona	Situation	Primary need	Failure to avoid
1	Meera, Pune family planner	MP-origin vegetarian, two children	Weekday relief and variety	Child-hostile spice/complexity
2	Kavita, Delhi joint family	Children + diabetic elder	Base meal with member add-ons	One “average” meal for all
3	Asha, Ahmedabad Jain couple	Strict exclusions	Trustworthy Jain safety	Onion/garlic leakage
4	Nandini, Chennai new mother	Postpartum support network	Gentle, culturally familiar plan	Medical claims
5	Shreya, Mumbai toddler parent	Unpredictable schedule	Fast fallback and kid format	Rigid weekly plan
6	Farah, Hyderabad family	Halal, mixed spice tolerance	Safe shared dinner	Treating halal as cuisine
7	Ritu, Jaipur homemaker	Budget-conscious, confident cook	Seasonal variety	Exotic expensive inputs
8	Sonali, Kolkata caregiver	Elder with soft-texture needs	Texture-aware add-ons	Conflating preference and safety
9	Lakshmi, Bengaluru migrant	Tamil identity, local exposure	Home-city blend	Stereotyped regionality

Each persona must map to test fixtures, not personal labels displayed to users.

8. Personas 10-17: individuals and shared households

#	Persona	Situation	Primary need	Premium trigger
10	Riya, Bengaluru professional	Lives alone, late work	20-minute single-serve meals	Automated groceries
11	Arun, Gurugram runner	Protein goal, Indian food	Macro-aware portions	Goal integrations
12	Vikram, Pune student	Shared flat, low skill	Cheap group meals	Unlikely; referral engine
13	Dev, Mumbai beginner	Wants to cook, fears failure	Confidence-ranked recipes	Guided cooking
14	Isha, Delhi creator	High novelty appetite	Discoverable but practical ideas	Advanced exploration controls
15	Karan, Hyderabad night-shift	Nonstandard meal timing	Circadian-aware slots	Schedule intelligence
16	Ananya, Kochi pescatarian	Seafood + freshness sensitivity	Local seasonal choices	Retail integration
17	Rohit, Indore shared couple	Alternating cooks	Cook-specific feasibility	Household collaboration

Design implications

- “Household” may contain one person.
- Serving size, leftover reuse, equipment, and effort must influence feasibility.
- Meal occasions cannot be locked to clock time.
- Beginner users need recipes and substitution support sooner than expert cooks.
- High novelty preference never permits unsafe or implausible exploration.

9. Personas 18-25: cultural and edge-case coverage

#	Persona	Situation	Critical requirement
18	Harpreet, Chandigarh family	High-energy breakfast tradition	Meal energy pattern by occasion
19	Saira, Lucknow fasting member	Periodic fasting in mixed household	Date-bound per-member overlay
20	Mohan, Patna-to-Mumbai migrant	Cost-sensitive regional comfort	Identity without ingredient impracticality
21	Teresa, Goa mixed-diet home	Vegetarian and non-veg members	Shared base + optional protein
22	Padma, Coimbatore elder couple	Low complexity and soft textures	Accessibility + portion realism
23	Neil, Indian diaspora London	Indian identity, local catalog	Locale availability layer
24	Sara, Dubai mixed-nationality couple	Cross-cultural compromise	Fairness-aware household blend
25	“Empty profile” new user	Minimal onboarding data	Safe, popular, diverse cold start

Persona acceptance protocol

Every release candidate runs golden journeys for all 25. For each day and meal slot, validation asks: Did the candidate pool survive? Were hard constraints respected? Was the cooking burden realistic? Was repetition controlled? Did the explanation use evidence actually present in the decision trace? Did a household minority become systematically ignored? Could the user recover from a wrong assumption in one action?

10. User psychology and behavior design

The product must work with five predictable psychological forces.

1. **Decision fatigue:** reduce choices to one strong default and a small alternative slate.
2. **Status-quo bias:** make the plan editable, but useful without editing.
3. **Ambiguity aversion:** explain feasibility and familiarity; avoid unexplained “AI magic.”
4. **Loss aversion:** “Never” must feel reversible from Settings; accidental rejection cannot silently erase a food forever.
5. **Reactance:** never moralize or lock users into a plan. Language is suggestive: “works well,” not “you must.”

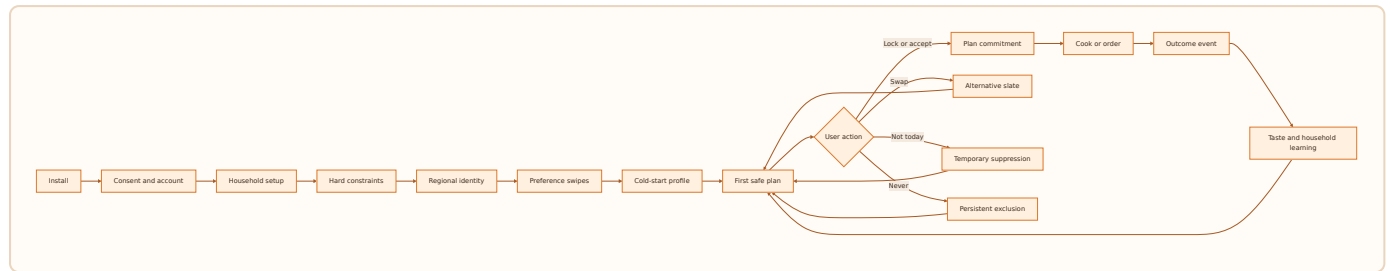
Household psychology

A planner, eater, shopper, and cook may be different people. Preferences therefore carry roles and confidence. A child’s dislike affects acceptance; a cook’s time constraint affects feasibility; an allergy affects safety; a guest preference affects one context. Treating all signals as identical votes creates bad plans.

Healthy engagement

The habit loop is cue → ready plan → small correction → completed meal → quiet learning. Streaks should reward useful outcomes (“four planned dinners completed”) without shaming missed days. Notifications are configurable and capped. Foofoo should celebrate regained time and household alignment, not calories avoided.

11. End-to-end lifecycle flow

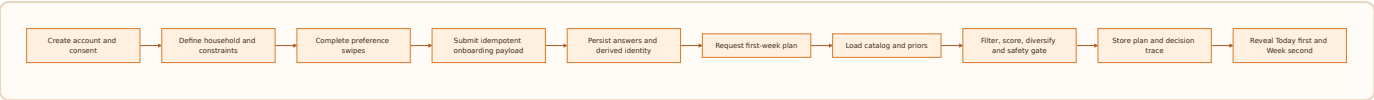


Lifecycle state model

anonymous → authenticated → consented → household_incomplete → onboarding_complete → cold_start → learning → mature → dormant → deletion_requested → purged.

State changes are explicit, versioned, observable, and recoverable. A recommender failure does not move a user backward; it produces a cached or popular-safe fallback. Consent withdrawal disables the affected learning path without breaking essential planning where feasible.

12. Journey A — first plan in under five minutes

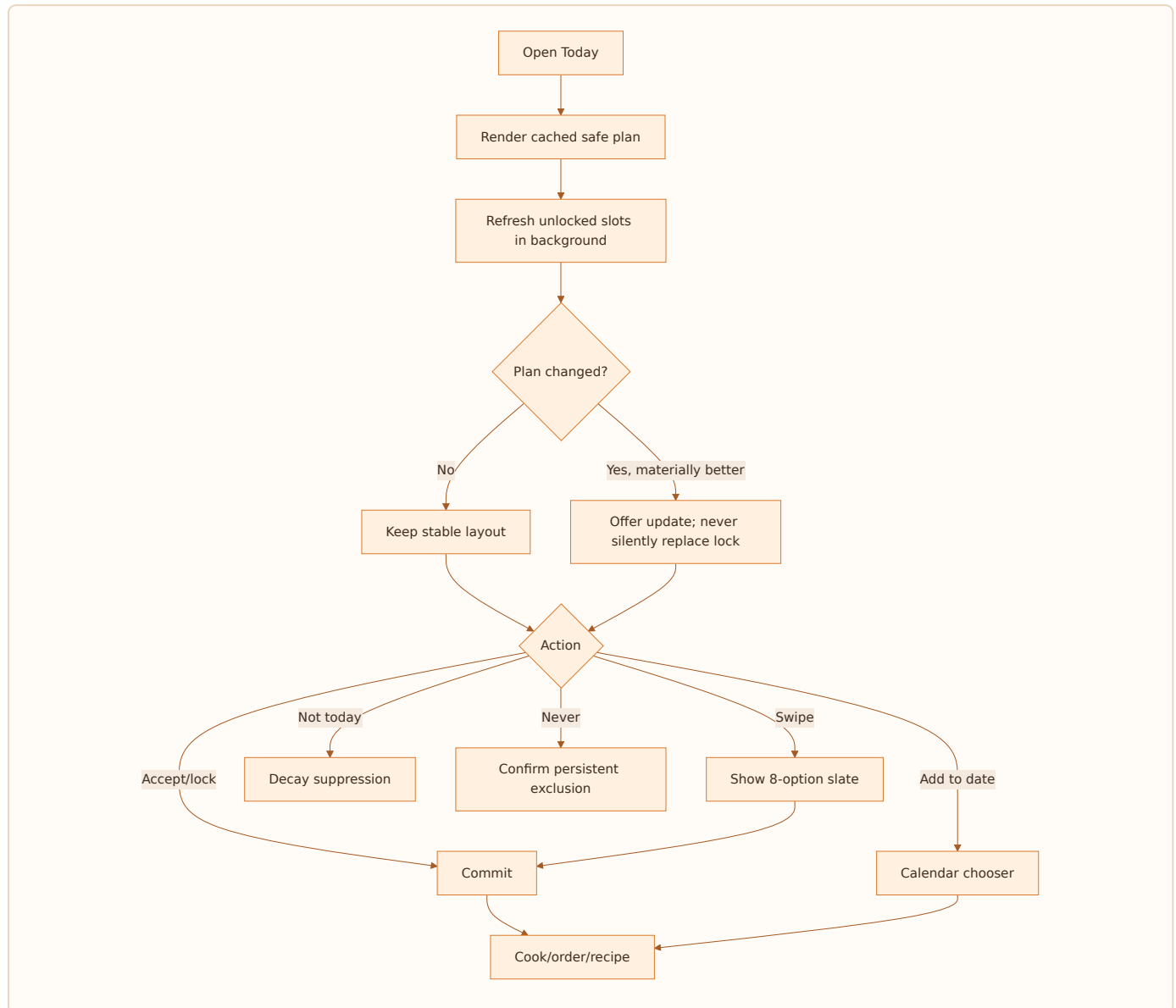


Success criteria: median completion under 4 minutes; 80% reach first plan; no more than one free-text field; users can skip non-safety questions; initial plan contains enough viable alternatives; source of every hard constraint is inspectable.

Recovery: interrupted onboarding resumes at the last completed screen. A user can correct a diet or allergen before plan generation. If a safety input changes later, all unlocked future slots are invalidated and regenerated; locked slots display a blocking review if now unsafe.

13. Journey B — daily decision and adaptation

The daily surface answers three questions in order: What is planned? Why does it fit? What can I do if it does not?



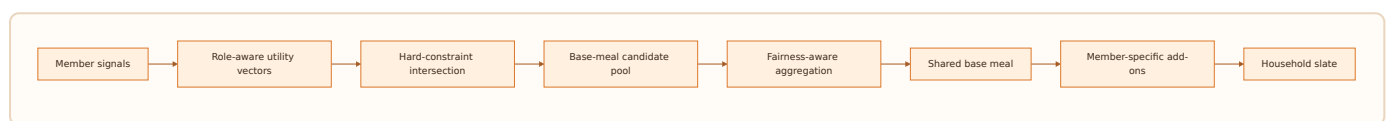
The layout remains stable during background refresh. Locked selections never change without direct user action. If connectivity fails, today's cached plan and previously loaded recipe remain usable; feedback queues locally with an idempotency key.

14. Journey C — household disagreement

Household collaboration is asynchronous and optional. The primary planner can share a lightweight voting link or invite another account. Foofoo aggregates preferences but does not force unanimity.

Decision policy

- Safety veto: any applicable hard constraint removes the candidate.
- Cook feasibility veto: if the assigned cook cannot execute it in context, it is removed or down-ranked.
- Strong persistent dislike: large negative preference, not necessarily a permanent exclusion unless marked Never.
- Child preference: influences acceptance and optional add-ons; does not override household nutrition or safety.
- Planner preference: gets a fairness floor so the person carrying the task is not always sacrificed.
- Occasion owner: birthday, fasting, or post-workout context may temporarily change weights.



Fairness is measured across weeks, not necessarily every meal. The engine should detect if one member's utility is persistently below a threshold and schedule compensating choices without violating constraints.

15. Information architecture

Tab	Primary purpose	Key objects
Today	Decide and act now	Meal cards, locks, alternatives, explanations
Week	Review and shape the plan	Seven days, slots, add-ons, refresh
Search	Intent-led override	Dish/class search, filters, add-to-date
Groceries	Convert plan into inputs	Consolidated ingredients, pantry exclusions
Profile	Household truth and control	Members, diet, allergens, region, Never list, privacy

Navigation rules

- Today is the default authenticated route.
- Deep links preserve intent: a notification opens the relevant slot; a household invite opens membership review.
- A user can always reach constraints and the Never list within two taps from a recommendation.
- Search is an override path, not the home experience.
- Recipe and dish details are context sheets over the current plan, preserving place on dismissal.

Object hierarchy

Household → Week Plan → Day → Meal Slot → Slate → Dish/Combo → Recipe/Order path.

Each slot has status (draft , recommended , locked , completed , skipped), one selected item, zero or more alternatives, optional member add-ons, a context snapshot, a model version, and an explanation derived from scored evidence.

16. UX design system

The active visual explorer establishes a warm consumer system. The implementation should retain the visual intent while meeting accessibility and performance requirements.

Visual language

Token group	Direction
Brand	Warm saffron/orange accent used selectively for primary action and appetite cues
Surfaces	Cream/off-white base, white elevated cards, restrained borders
Type	Friendly geometric sans; clear hierarchy; minimum 16px body on mobile
Imagery	Realistic Indian home food, consistent crop, no misleading garnish
Motion	Spring-based swipes, immediate haptics, reduced-motion alternative
Status	Green for safe/confirmed, amber for attention, red only for destructive/safety states

Accessibility

WCAG 2.2 AA contrast; 44×44 minimum targets; screen-reader labels that include dish, slot, status, and action; no color-only meaning; dynamic type to 200%; captions for instructional video; logical focus order; accessible confirmation for Never and deletion.

Copy principles

Concise, household-aware, and nonjudgmental. “Not right for today?” beats “Reject.” “Because it is quick and your family often likes similar meals” beats “92% match.” Safety explanations name the constraint; personalization explanations never reveal another member’s sensitive health data.

17. Screen specification — access and consent

ID	Screen	Required elements	Primary action	States / acceptance
AU-01	Splash/restore	Logo, local session restore, health probe	Automatic	Cached route <2s; no blocking animation
AU-02	Welcome	Promise, food imagery, sign in/create	Create account	Value understood without carousel
AU-03	Sign up	Email, password, age gate, terms links	Continue	Inline validation; no sensitive food data yet
AU-04	Sign in	Email/password, reset	Sign in	Generic auth errors; rate-limited
AU-05	Reset password	Email and confirmation	Send link	Does not expose account existence
CO-01	Consent	Essential personalization, analytics, notifications, retention controls	Save choices	Granular, versioned, optional items off by default where required
CO-02	Privacy summary	Plain-language purposes, export/delete links	Continue	Accessible before consent

Analytics: `welcome_viewed`, `signup_started`, `signup_completed`, `signin_failed`, `consent_viewed`, `consent_changed` with policy version. Never put email, names, health conditions, or free text in analytics properties.

18. Screen specification — onboarding I

ID	Screen	Inputs	Logic	Completion rule
OB-01	Planner identity	Display name; who usually plans/cooks	Role initialization	Name optional; role required
OB-02	Household type	Solo, couple, family, joint/shared	Determines branch, not persona label	One selection
OB-03	Members	Age band, eating role, optional nickname	Create member rows	At least one eater
OB-04	Regional identity	Home state/culture, current city, time since migration	Blend home and local priors	Home may be skipped; city required for context
OB-05	Diet	Veg, non-veg, egg, vegan, jain; per-member overrides	Hard constraint intersection	Explicit confirmation
OB-06	Allergens	Standard list + “none known” + uncertainty note	Bitset/relations; safety gate	Must actively choose none or items

Critical UX rule: diet and allergen screens explain household-wide impact. Changing them later triggers future-plan review. No inferred allergy is silently promoted to a hard constraint.

19. Screen specification — onboarding II

ID	Screen	Inputs	Model effect	UX requirement
OB-07	Cook capability	Beginner/intermediate/advanced, equipment, weekday minutes	Feasibility prior	Examples make levels concrete
OB-08	Preference swipes	8–12 class/dish cards	Initializes class and genome affinities	Yes/no buttons mirror gestures
OB-08b	Context preferences	Budget, novelty, health orientation, notification time	Soft weights	Everything skippable except time zone
OB-09	Review	Household facts and safety constraints	Final confirmation	Edit links by section
OB-10	Plan generation	Progress with useful copy	Calls recommendation	Retry and safe fallback
OB-11	First-plan reveal	Today’s plan and three teaching cues	Activation	User can lock or swap immediately

Onboarding uses progressive disclosure. The engine records source (`explicit_onboarding`) and confidence for every derived attribute. Skipped answers reduce confidence and widen exploration; they never create false precision.

20. Screen specification — Today and alternatives

ID	Screen	Required elements	Actions	Edge states
P-01	Today	Date, meal slots, selected dish/combo, add-ons, explanation chip	Lock, swap, detail, add-to-date	Cached/offline banner; partial slots
P-02	Alternative carousel	8 candidates, image, time, fit tags, position	Accept, next, not today, never	Exhausted slate → broaden intent
P-03	Dish detail	Components, time, difficulty, region, nutrition range, allergens, why	Recipe, lock, add to date, order	Missing recipe; data-confidence badge
P-04	Explanation sheet	Top positive reasons, trade-off, safety confirmation	Feedback on reason	Never show raw weights or sensitive member condition
P-05	Lock confirmation	Slot and selection	Lock/unlock	Concurrent update handled
P-06	Never confirmation	Dish, consequence, undo route	Confirm Never	Distinguish from Not Today

The eight-option slate is stable for a decision session so analytics can reconstruct exposure. Reopening a slate does not reshuffle unless the user explicitly refreshes.

21. Screen specification — Week, search, and action

ID	Screen	Required elements	Actions	Acceptance
W-01	Week plan	7-day grid/list, slot statuses, locks	Edit slot, refresh unlocked, share	Locked items preserved
W-02	Selective refresh	Scope summary and reason	Refresh	Shows what will/not change
W-03	Add to date	Calendar, slot, replace/add behavior	Confirm	Conflict explicit
S-01	Search	Query, recent intent, filters	Open/add	Results <500ms target
S-02	Filters	Diet-safe fixed, cuisine, time, difficulty, meal role	Apply	Cannot disable hard constraints
R-01	Recipe	Ingredients, steps, servings, substitutions, timers	Start cooking	Offline after first load
O-01	Cook or order	Recipe path, partner deep links, transparent sponsorship	Choose	Commercial content labelled

Search results use the same hard-constraint service as recommendations. A typed request for an unsafe dish produces a respectful block and safe alternatives, not a silent empty result.

22. Screen specification — groceries, household, privacy

ID	Screen	Required elements	Actions
G-01	Grocery list	Aggregated quantities, categories, source slots	Check, exclude pantry, share
G-02	Pantry preferences	Staples, avoid auto-add, last reviewed	Update
H-01	Household	Members, roles, invite status	Add/edit/remove
H-02	Member profile	Diet, allergens, preference visibility, temporary context	Save
H-03	Invite	Link/code, permissions, expiry	Send/revoke
PR-01	Profile	Region, cook ability, notification, language	Edit
PR-02	Never list	Excluded dishes/classes, date, source	Restore
PR-03	Data and privacy	Consents, export, delete, retention summary	Request
PR-04	Recommendation controls	Familiarity/novelty, planning effort, explanation detail	Save

Removing a member starts a review before deleting member-linked preference evidence. Household administrators cannot see another adult’s sensitive condition unless that person chooses to share it; the recommender may use a privacy-preserving “constraint applies” flag.

23. Functional requirements

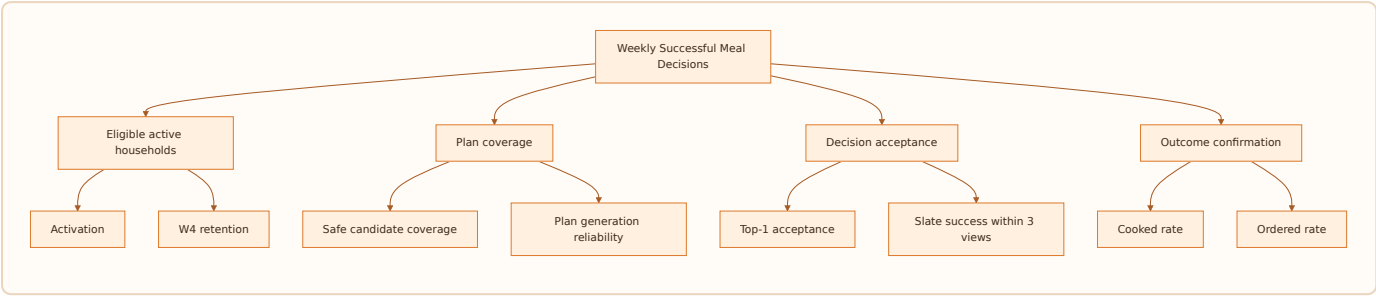
ID	Requirement	Priority	Acceptance evidence
FR-01	Generate a 7-day, multi-slot household plan	P0	Contract + golden personas
FR-02	Enforce diet, allergen, religious, occasion, Never constraints twice	P0	Zero-row safety queries
FR-03	Return 8 stable alternatives per eligible slot	P0	Deterministic session test
FR-04	Lock selections across refresh	P0	Concurrency test
FR-05	Process Not Today with time decay and Never persistently	P0	State transition tests
FR-06	Learn from exposure, action, and outcome events	P0	Feature update trace
FR-07	Generate base meal plus member add-ons	P1	Joint-family fixtures
FR-08	Search and add a safe dish to a date	P1	UI/API integration
FR-09	Explain each recommendation from actual contributions	P1	Trace-to-copy test
FR-10	Build a grocery list from committed plan	P1	Unit conversion tests
FR-11	Offline-display cached plan and queue feedback	P1	Airplane-mode journey
FR-12	Export and delete user data	P0	End-to-end compliance test
FR-13	Support model/config version and experiment assignment	P0	Decision trace
FR-14	Provide cook/order routes without paid-rank contamination	P2	Marketplace policy test

24. Non-functional requirements

Domain	Target
Safety	Zero known hard-constraint violations; gate blocks the slate, not just the item
Availability	99.9% monthly for plan read; safe cached fallback during RE outage
Latency	cached today p95 <1s; fresh plan p95 <3s; server recommendation p95 <800ms target
Scale	horizontal stateless APIs; partition high-volume events; 10× demand headroom before redesign
Privacy	data minimization, purpose-bound consent, export/delete, no PII in analytics
Security	JWT validation, household ownership checks, RLS, private RE schema, secret rotation
Accessibility	WCAG 2.2 AA and screen-reader-complete primary journeys
Reliability	idempotent mutations and events; retries bounded; dead-letter visibility
Explainability	every result stores candidate set, filters, signal contributions, version, rank
Reproducibility	offline replay within numerical tolerance using trace snapshot
Cost	model and infra cost per WSMD tracked; LLM never on synchronous ranking hot path
Freshness	catalog publishing SLA and feature staleness indicators

Current-state note (repository, 4 Aug 2026): deployed P0 backend and mobile core exist; local release candidate includes search/filter, weather, explanations, MMR, offline evaluation, and a bounded graph layer. The current status reports a physical-device push-delivery test and broader food-knowledge depth as remaining work. These are verification claims from active internal documents, not independently re-audited in this PRD.

25. Success metrics and metric tree



Metric definitions

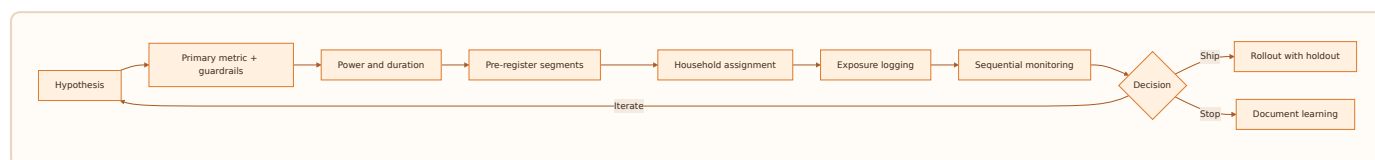
Metric	Definition
Activation	Household completes onboarding and accepts/locks ≥ 2 slots within 24h
Top-1 acceptance	Recommended first items accepted $\div$ valid first-item exposures
Slate success	Sessions with an accepted item within first 3 impressions $\div$ slate sessions
Plan adherence	Completed intended slots $\div$ committed eligible slots
Decision time	Active seconds from plan view to commitment, excluding background
Variety satisfaction	Accepted slots meeting policy with no repetition complaint event
Safety incident rate	Confirmed constraint violations per million slates; target 0
Household fairness	10th-percentile member utility over rolling 28 days
W4 retained household	≥ 3 WSMD in week four
Recommendation regret	Share of accepted items replaced before the meal window

Guardrails accompany every growth or ranking experiment: safety, complaint rate, Never rate, latency, cook completion, and 7/28-day retention.

26. Experimentation framework

Experiments operate at the household level to prevent cross-member contamination. Assignment is deterministic (`hash(household_id, experiment_id)`) and stored before exposure. Analysis is intention-to-treat; triggered analysis may supplement but never replace it.

Experiment lifecycle



Required analyses

- Overall and pre-registered segments: cold-start/mature, solo/household, region, diet, cook skill, connectivity tier.
- Novelty effects over at least 14 days; one-day clicks are insufficient.
- Interference review where invited members interact with the same plan.
- CUPED or pre-period covariates for mature households where appropriate.
- False-discovery control for many secondary metrics.
- Long-lived 1-5% global holdout to measure personalization's cumulative value.

No experiment may weaken a safety filter. Exploration is only within the already-safe pool.

27. Growth strategy

Growth begins with product utility, then household network effects, then ecosystem distribution.

Loops

1. **Relief loop:** strong plan → less decision time → repeat morning open → better evidence → stronger plan.
2. **Household loop:** planner invites eater → more preference evidence → fewer complaints → planner retains → more invitations.
3. **Plan-sharing loop:** share a useful weekly plan → recipient sees concrete value → installs with locale context.
4. **Recipe loop:** accepted decision leads to recipe completion → outcome confidence improves → search acquisition grows.
5. **Partner loop:** grocery/order conversion proves intent → partner improves availability data → plans become more actionable.

Growth requirements

- Invite without forcing account creation before the recipient understands the purpose.
- Deep links preserve household and plan context with expiry and abuse controls.
- Referral incentives reward activation or WSMD, not raw installs.
- ASO content targets the pain phrase and regional/household use cases.
- Lifecycle messaging pauses when no new value exists; max caps are enforced.
- Win-back shows improved plan quality or a relevant context, not generic urgency.

Growth analytics must separate planners, eaters, and invited non-planners; one household is not counted as multiple independent retained customers.

28. Go-to-market strategy

Phase 1 — Design partners

Recruit 100–300 households across Pune, Bengaluru, Delhi NCR, Mumbai, Hyderabad, Ahmedabad, Chennai, and one Tier-2 cluster. Balance household types and regional origins. Founders observe onboarding and the morning decision weekly. Success is qualitative trust plus 3+ WSMD/week, not downloads.

Phase 2 — City-cluster launch

Launch through apartment communities, parent groups, workplaces, fitness coaches, and regional food creators. Use invitation cohorts to maintain catalog coverage and support quality. Messaging leads with relief; AI is proof, not headline.

Phase 3 — Public India launch

Combine ASO, vernacular content, referral incentives, creator demonstrations, and partnerships. Build a wait-free self-serve onboarding only after cold-start coverage is stable.

Phase 4 — Monetization and partners

Introduce premium household controls, advanced planning, recipes, nutrition views, and integrations. Retail/order partners receive explicit downstream conversion opportunities; no unlabelled paid insertion into organic rank.

GTM scorecard

CAC per activated household, activation, W1/W4 retention, WSMD, organic share, invite acceptance, support tickets per 100 households, catalog coverage by cohort, and cost per successful decision.

29. Monetization

Recommended ladder

Tier	Illustrative offering	Pricing test
Free	Today/week plan, core swipes, basic household, limited history	₹0
Plus	Advanced household controls, full recipes, grocery automation, unlimited swaps, deeper explanations	₹99-149/month
Family Pro	More members, goal overlays, shared planning, integrations, priority support	₹199-299/month
Partner/API	Food intelligence and qualified intent services	Contracted usage/value

Pricing values are hypotheses. Test willingness to pay only after habit formation and clear feature value. Use UPI AutoPay and app-store billing as applicable; publish renewal terms plainly.

Monetization principles

- Core safety is never paywalled.
- Free plans remain genuinely useful.
- Sponsored inventory is labelled and ranked only after safety; users can disable commercial suggestions.
- Health-oriented premium features require evidence and appropriate professional review.
- Subscription cancellation and data deletion are simple.
- Optimize lifetime successful decisions and trust, not short-term conversion.

Potential future revenue includes retailer affiliate fees, order referral, branded but transparent recipe placements, appliance integrations, employee wellness, and B2B recommendation APIs. Each requires a conflict-of-interest review.

30. Product roadmap and future vision

Version	Product promise	Intelligence	Data/Platform
v1	Safe plan that is better than deciding alone	Rules, cohort/class priors, content match, MMR	Canonical catalog, events, traces
v2	Learns an individual and household	Taste vectors, decay, contextual bandit	Feature store, experiments, quality dashboards
v3	Plans composition and groceries	Knowledge-graph retrieval, leftovers, substitution	Graph projections, inventory/retail adapters
v4	Anticipates context and long-term balance	Sequence models, constrained weekly optimization	Online/offline feature parity, shadow deploys
v5	Food operating system	Multi-objective policy, federated/causal learning where justified	Multi-region, locale packs, partner platform

Future vision: Foofoo becomes a private household food memory. It understands what people enjoy, can safely eat, can realistically make, already have, recently consumed, and may need tomorrow. It coordinates cooking, shopping, ordering, appliances, and health services without allowing any partner to own or distort the household’s preferences.

The sequence is intentionally trust-first. A beautiful graph or sophisticated model has no value if the household’s first week contains implausible meals.

Part II — FooFoo Recommendation Engine Bible

31. Recommender objectives and invariants

The recommender maximizes expected household meal utility subject to safety, feasibility, diversity, fairness, and system constraints.

For household h , context c , meal slot s , and candidate d :

```
maximize  $E[U(h,d,s,c)] + \text{exploration}(d,h) - \text{burden}(d,c) - \text{repetition}(d,h)$ 
```

subject to:

- $\text{Safe}(d,h)=1$
- $\text{OccasionFit}(d,s)=1$
- $\text{Never}(d,h)=0$
- plan composition constraints
- latency and candidate-coverage minimums.

Non-negotiable invariants

1. Filters precede scoring; safety gates run again after re-ranking.
2. Unknown safety data is not treated as safe for high-risk cases.
3. Every exposure has a stable request, slate, item, rank, model, and experiment identifier.
4. The LLM may enrich or explain offline; it cannot invent a dish into the live safe pool.
5. A locked plan slot is immutable except by the user or an explicit safety invalidation.
6. Personalization confidence controls how much the system departs from cohort priors.
7. No single click permanently rewrites taste; explicit Never is the exception.

32. Multi-stage recommendation pipeline



Stages and budgets

Stage	Role	Target p95
Snapshot	Assemble immutable request context	80ms
Class planning	Choose meal roles before dishes	60ms
Retrieval/filter	50-500 safe candidates	180ms
Features	Batch online feature read	100ms
Score/aggregate	Score all candidates	120ms
Re-rank/optimize	Diversity + plan constraints	150ms
Gate/explain/persist	Verify and emit	110ms

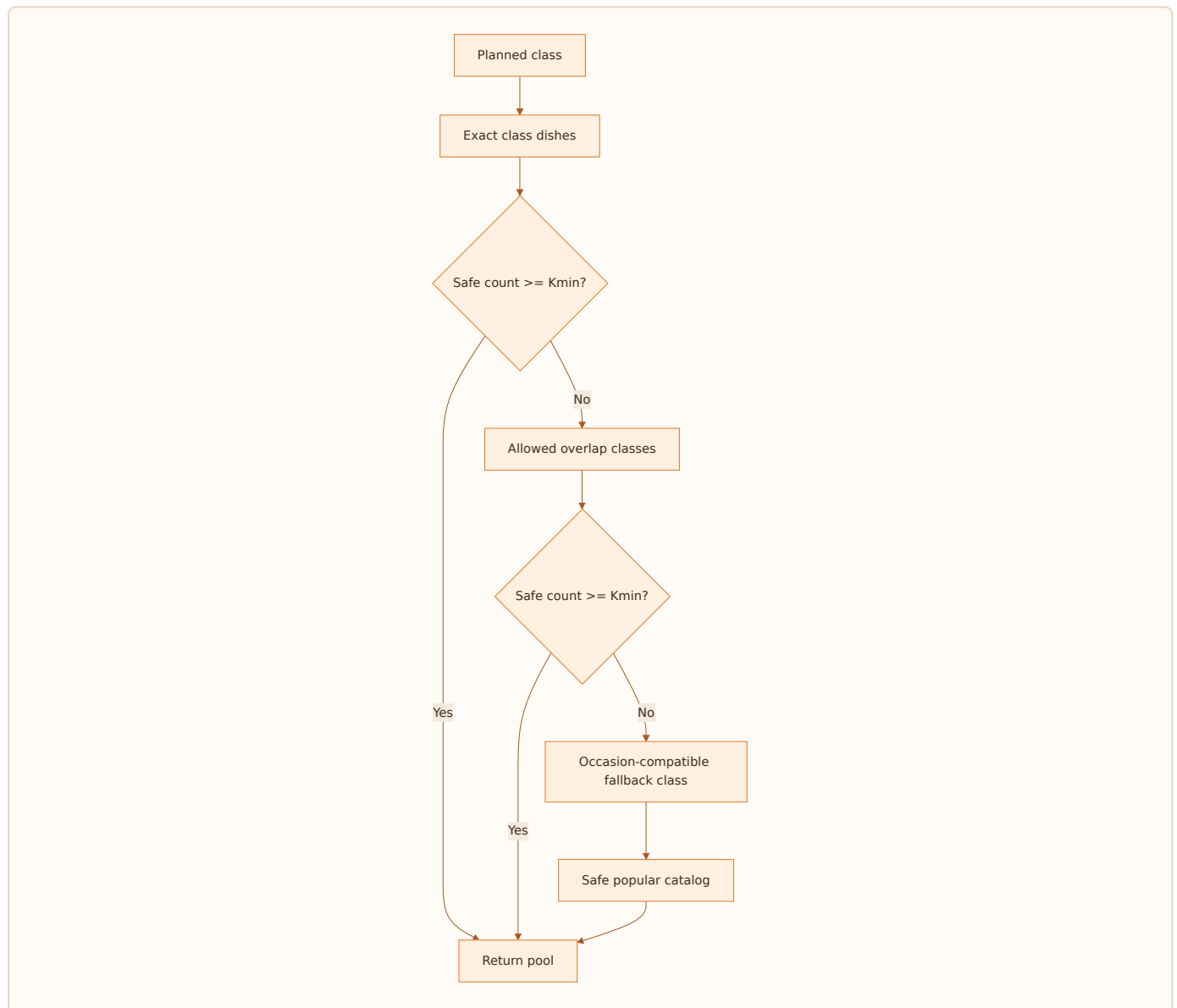
Budgets are targets, not promises. The request degrades through named fallbacks: live personalized → cached features → cohort priors → safe popular plan → last known safe plan.

33. Candidate generation

Candidate generation is recall-oriented. Multiple generators produce `(dish_id, generator, retrieval_score)` tuples:

- class-to-dish mappings from the planned meal class;
- regional/home-state affinity;
- current-city exposure;
- household favorites and similar-genome neighbors;
- seasonal/weather-compatible foods;
- underexposed safe catalog for exploration;
- leftovers and ingredient reuse (v3);
- substitution graph paths (v3);
- collaborative cohort neighbors after scale (v3+).

Candidates are unioned, deduplicated by canonical dish ID, and filtered. Generator contribution is logged for later attribution. A minimum coverage policy broadens the class in a controlled hierarchy if fewer than `K_min` candidates survive. It never drops a hard constraint.



Coverage gaps are operational data with region, class, constraints, and failed counts; they drive content acquisition.

34. Hard constraints and safety gates

The candidate set is:

$$C_safe = C \cap C_diet \cap C_allergen \cap C_religion \cap C_occasion \cap C_never \cap C_availability.$$

Diet and Jain eligibility are derived from ingredient ground truth. Allergen filtering traverses dish ingredients rather than trusting a display-level cached flag. Household constraints are the union of applicable eater constraints for a shared base dish; member add-ons use that member’s constraints plus contamination policy.

Gate behavior

Gate	Failure action
Diet	Remove candidate; block final slate if leaked
Allergen	Remove candidate; raise incident if final leak
Religious/Jain	Remove candidate; block slate
Meal role/composition	Repair slot or regenerate
Never	Remove; explicit restore required
Missing provenance	Quarantine content from high-risk surfaces

Safety regression uses adversarial fixtures: ingredient aliases, compound ingredients, optional garnish, cross-diet variants, missing data, and substitutions. A post-rank gate verifies the exact selected dish version and components, not merely the parent dish.

35. Feature model

Features are grouped by source and refresh cadence.

Family	Examples	Cadence
Household	composition, diet intersection, cook role	change-driven
Taste	class affinity, genome vector, dish history	near-real-time
Content	genome tags, ingredients, region, difficulty	publish-time
Context	slot, weekday, weather, festival, time budget	request-time/cache
Plan	recent classes, ingredients, methods, colors, burden	request-time
Popularity	impression-normalized acceptance, cook completion	daily/hourly
Quality	catalog confidence, recipe completeness, image quality	publish-time
Exploration	impression count, posterior uncertainty	event-driven

Features have an owner, definition, data type, null policy, valid range, timestamp, source lineage, and offline/online transformation parity test. Null is never casually coerced to zero; it maps to a documented default and missingness indicator where informative.

36. Base scoring equation

For candidate d :

$$z(d) = w_c C(d) + w_g G(d) + w_h H(d) + w_x X(d) + w_q Q(d) + w_e E(d) - P(d)$$

$$p_{\text{accept}}(d) = \sigma(z(d)) = 1 / (1 + e^{-z(d)})$$

where:

- C : cohort/class prior;
- G : Meal Genome/content match;
- H : personal and household history;
- X : context fit;
- Q : content quality and feasibility;
- E : bounded exploration bonus;
- P : repetition, burden, temporary suppression, and uncertainty penalties.

Weights interpolate with personalization confidence $p \in [0, 1]$:

$$w_i(p) = (1-p) w_{i,\text{cold}} + p w_{i,\text{mature}}.$$

This ensures cold users lean on priors while mature users lean on evidence. Scores are calibrated against acceptance outcomes; rank order alone is insufficient for product decisions such as confidence labels or notification thresholds.

37. Personal history and time decay

Event evidence decays so recent behavior matters more without erasing durable taste:

$$H_d = \sum_j a(\text{type}_j) \cdot \exp(-\lambda \Delta t_j) \cdot \text{position_debias}(j) \cdot \text{context_similarity}(j).$$

Illustrative event signs: lock/cooked/rated-positive > accept > detail-view; passive impression is neutral; swipe-past is weak negative; Not Today is strong temporary negative; Never is a hard exclusion. Exact values live in versioned configuration and are learned from outcomes.

Exposure correction

An unshown dish cannot be considered disliked. Propensity or position correction prevents the first-ranked items from acquiring self-reinforcing popularity simply because they received more exposure. Training data stores the full slate and selection propensity where exploration policy permits.

Repetition penalties

Penalty components include same dish, parent dish, main ingredient, meal class, cooking method, cuisine, texture, and color family over separate windows. Explicit comfort-food preference and planned leftovers can relax—but not erase—specific penalties.

38. Household intelligence mathematics

Let each member m have predicted utility $u_m(d)$, role weight r_m , and confidence q_m . A naive mean can repeatedly ignore a minority. Foofoo uses a blended social-welfare objective:

$$U_{\text{house}}(d) = \alpha \sum_m \hat{w}_m u_m(d) + (1-\alpha) \min_m u_m(d) - \beta \text{Burden}(d, \text{cook})$$

where $\hat{w}_m = r_m q_m / \sum_j r_j q_j$. The minimum-utility term provides a fairness floor. Applicable safety constraints are not utilities; they already removed candidates.

For a shared base plus add-ons:

$$U_{\text{bundle}} = U_{\text{house}}(\text{base}) + \sum_m U_m(\text{addon}_m \mid \text{base}) - \gamma \text{Complexity}(\text{bundle}).$$

Complexity penalizes extra burners, distinct ingredient sets, extra active minutes, and coordination. This prevents “personalization” from turning one meal into five separate meals.

Role examples

The primary cook’s time feasibility receives high weight; an eater’s favorite has preference weight; a temporary guest has contextual weight; a toddler signal is smoothed to avoid a week of only favorite foods. Roles are policy, transparently tested for systematic unfairness.

39. Meal Genome similarity

Each dish has sparse categorical tags plus a dense vector. Household taste is an attention-weighted aggregate of accepted dish vectors and explicit class signals.

$$\text{sim}(d, h) = \cos(v_d, v_h) = (v_d \cdot v_h) / (||v_d|| \cdot ||v_h||).$$

For mixed feature families:

$$G(d, h) = \sum_k \eta_k \text{sim}_k(d, h),$$

where a categorical family can use weighted Jaccard and numeric nutrition/time features use normalized distance.

The vector space must not collapse safety and taste. Diet/allergen/religious data remain constraints. Genome dimensions represent sensory and practical similarity: base ingredient, cooking method, texture, flavor, richness, meal role, cuisine, time, equipment, serving format, temperature, season, accompaniment, familiarity, and nutrient bands.

Embedding models may propose similarities, but curated ontology edges and human review anchor high-impact relations.

40. Diversity and MMR ranking

After pointwise scoring, Maximum Marginal Relevance selects a slate:

$$\text{MMR}(d) = \lambda \text{Rel}(d) - (1 - \lambda) \max_{s \in S} \text{Sim}(d, s).$$

S is the already selected set. Similarity combines genome, main ingredient, meal class, cooking method, and cuisine. The penalty differs by surface: a one-slot carousel needs visually and conceptually distinct choices; a week plan needs balanced continuity and ingredient reuse.

Diversity policy

- Never sacrifice hard safety.
- Preserve the best high-confidence item at rank 1 unless weekly optimization changes it.
- Cap near-duplicate parent variants in one slate.
- Enforce minimum class/cuisine/method coverage when the safe pool allows.
- Record the relevance loss paid for diversity.
- Measure satisfaction and completion, not only unique tags.

MMR is deterministic for a stored seed and candidate set, supporting replay. Exploration randomization occurs through a logged policy before or within constrained re-ranking.

41. Weekly constrained optimization

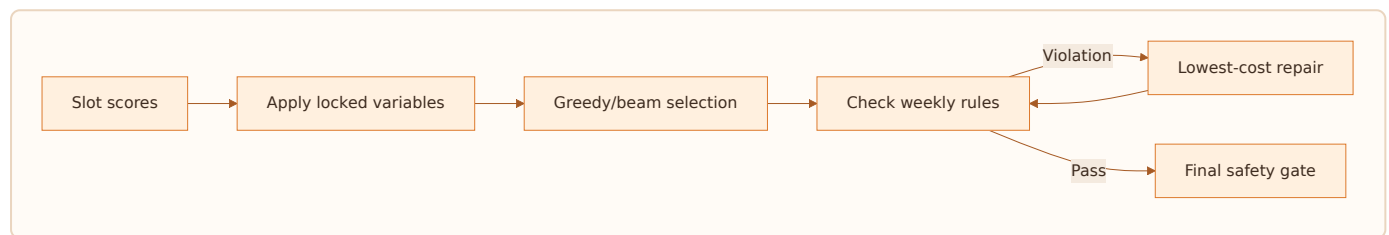
A weekly plan is not seven independent rankings. Let binary $x_{\{d,s\}}$ mean dish d is selected for slot s :

maximize $\sum_{\{d,s\}} x_{\{d,s\}} \text{Score}(d,s) - \text{repetition} - \text{burden} + \text{reuse} + \text{balance}$

subject to $\sum_d x_{\{d,s\}}=1$ for each active slot, hard safety, locks, class composition, cook-time budgets, and policy limits.

Practical solver strategy

v1 uses greedy slot planning plus MMR and repair. v2 adds beam search over days. v3 may use integer programming for a weekly bundle when inventory, nutrition ranges, and leftovers are sufficiently trustworthy. Locked slots become fixed variables. A selective refresh optimizes only unlocked variables around them.



42. Cold start algorithms

Cold start has four cases: new household, new member, new dish, and new region.

New household

1. Apply explicit hard constraints.
2. Build composable cohort priors from household, region, cook skill, and time.
3. Convert 8–12 onboarding swipes into class/genome evidence.
4. Set confidence p from completeness and signal agreement.
5. Mix popular-safe exploitation with bounded, diverse exploration.

```
Score_cold = (1-p) Prior_cohort + p SwipeTaste + Context + Quality - Penalties .
```

New dish

Use catalog quality, class mapping, genome similarity, regional evidence, and conservative exploration. Never use raw popularity of unexposed content as zero-quality proof.

New region

Fall back through state → culinary zone → national occasion-compatible priors, while logging a coverage gap.

Exit

Exit cold start when effective evidence—not raw event count—crosses a configurable threshold and calibration is stable. Ten repeated impressions are not ten independent preference signals.

43. Contextual bandits and exploration

After safe candidate generation, a contextual bandit can choose limited exploratory exposure. Thompson Sampling maintains a posterior per dish/class/context; LinUCB or a neural contextual bandit may follow when data supports it.

For Beta-Bernoulli Thompson Sampling:

$\theta_d \sim \text{Beta}(\alpha_d, \beta_d)$ and select among safe candidates using θ_d blended with rank score. Success may be cooked/accepted; failure definitions must account for missing outcome and position.

Guardrails

- Exploration share capped by confidence and user novelty preference.
- No exploration on safety uncertainty.
- Do not explore high-burden dishes on time-pressured weekdays.
- Household fairness constraints remain active.
- Store propensity for unbiased evaluation.
- Use a stable control policy and off-policy evaluation before rollout.

The bandit is not a substitute for a correct catalog or base ranker. It improves evidence allocation within a safe, high-quality pool.

44. Ranking model roadmap v1→v5

Version	Model	Training data	Promotion gate
v1	Configured linear score + MMR	Priors, explicit swipes, rules	Safety + golden personas + offline sanity
v2	Calibrated logistic/GBDT ranker	Impressions, accepts, cooks, context	AUC/NDCG plus online WSMD lift
v3	Two-tower retrieval + GBDT/deep ranker	Larger exposure corpus, content embeddings	Retrieval recall + diversity + fairness
v4	Sequence-aware household model	Ordered sessions/weeks	Long-term retention and regret reduction
v5	Constrained policy optimization	Causal/experimental outcomes	Off-policy safety, long-horizon WSMD

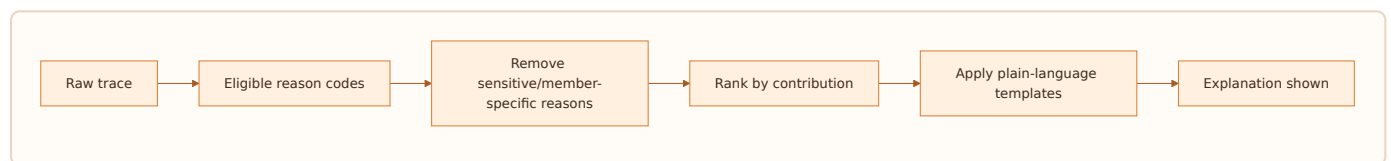
Model sophistication is earned by sample size and operational maturity. Each version retains deterministic rule fallbacks. Champion/challenger runs in shadow first; discrepancies are inspected by cohort and constraint type. Promotion requires calibrated improvements in real outcomes, not only offline ranking metrics.

45. Explanations and decision traces

Every recommendation stores:

- request, household snapshot hash, context snapshot;
- planned class and candidate generators;
- candidates before/after each filter with reason codes;
- feature version and values used;
- signal contributions, penalties, point score;
- household aggregation and diversity deltas;
- final rank, safety gate result, model/config/experiment versions;
- response and subsequent events.

User explanation is a faithful compression: select the top 2–3 positive, non-sensitive contributions that pass copy rules. Example: “Quick for a weekday, familiar to your household, and different from yesterday’s main ingredient.”



If no defensible reason exists, show a neutral label such as “A safe popular option for this meal,” not fabricated personalization.

46. Offline evaluation

Datasets

- Time-based train/validation/test split.
- Household-level split for generalization checks.
- Cold-start slice, mature slice, sparse regions, strict diets, large households.
- Counterfactual evaluation subset from randomized safe exploration.

Metrics

Retrieval Recall@K, NDCG@K, MRR, calibration error, coverage, catalog exposure Gini, intra-list diversity, repetition violations, constraint violations, inference latency, and estimated WSMD uplift.

Replay requirements

Given a trace snapshot, the engine reproduces candidate filters and ranks within a numeric tolerance. Feature leakage tests prevent use of future outcome information. Offline gains must hold across high-risk cohorts. A model that improves average NDCG while degrading Jain or allergen-safe coverage is rejected.

Human review

Food experts and representative users evaluate paired plans for plausibility, composition, cultural fit, burden, and explanation truthfulness. Human preference is evidence, not an unquestioned label; reviewer agreement is measured.

47. Online monitoring and model governance (operational, not policy paperwork)

Operational dashboards show request rate, p50/p95/p99 latency, fallback share, candidate count after each filter, safety gate failures, top-1 acceptance, slate success, WSMD, Never/Not-Today rates, drift, and feature freshness.

Drift detection

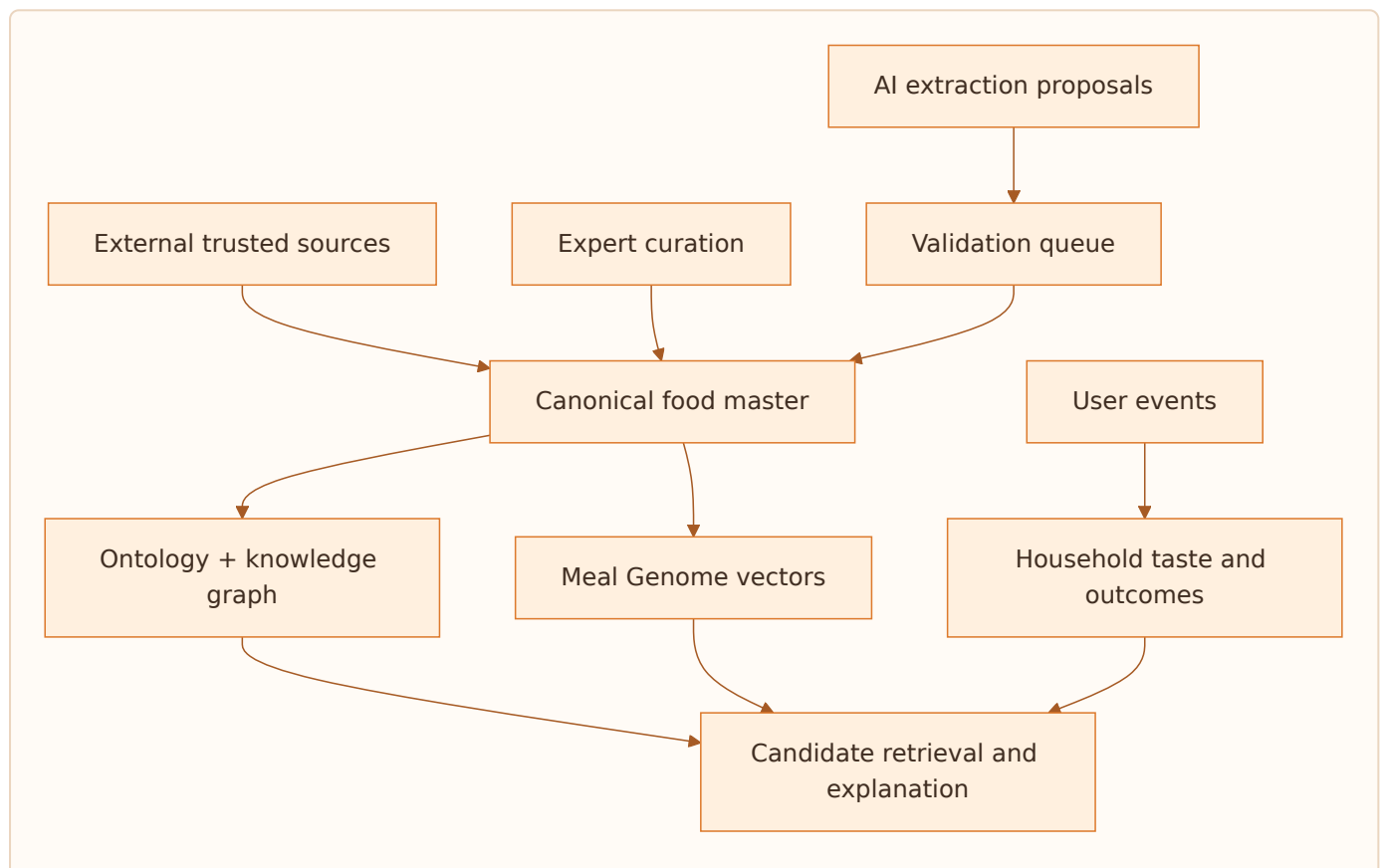
- Population drift in household/context features (PSI/Jensen-Shannon).
- Prediction drift and calibration by cohort.
- Outcome drift by region, season, and meal slot.
- Catalog drift: untagged or low-confidence new content.
- Feedback drift caused by UI or event-contract changes.

Alerts link to traces, model version, deploy, and catalog publish. Rollback changes the active model/config pointer; it does not require a mobile release. A kill switch disables learned ranking and serves safe priors. Safety incidents trigger candidate quarantine and trace preservation.

Part III — Food Intelligence Bible

48. Food intelligence architecture

Food intelligence separates canonical facts, culturally contingent knowledge, derived signals, and user evidence.



Canonical names and IDs are stable. Regional labels, recipes, nutrients, availability, and substitutions are versioned assertions with provenance and confidence. AI can propose; publishing high-impact facts requires deterministic validation and, for nutrition/safety, trusted source or qualified review.

49. Ontology domains

Domain	Key concepts
Dish identity	canonical dish, variant, alias, translation, parent
Ingredient	canonical ingredient, form, preparation, category, allergen
Meal role	breakfast, lunch, dinner, snack, tiffin, accompaniment, beverage
Composition	base, main, side, bread, rice, dal, salad, condiment, dessert
Cuisine/region	country, state, culinary region, community, migration affinity
Technique	boil, steam, sauté, fry, bake, ferment, pressure cook
Sensory	flavor, texture, richness, temperature, spice
Practical	time, skill, equipment, cost band, batchability, leftover quality
Nutrition	serving, macro/micro ranges, source and confidence
Suitability	diet, religious, age/texture, contextual—not diagnostic claims
Temporal	season, festival, fasting window, weekday/weekend
Relationship	pairs-with, substitutes, variant-of, contains, typical-in

Ontology terms have canonical codes, display labels, synonyms, locale, hierarchy, definition, source, confidence, effective dates, and status. Labels may change; codes and meaning do not change silently.

50. Meal Genome

The Meal Genome is a multi-family representation, not one opaque embedding.

Proposed dimensions

1. Meal role and component role
2. Primary ingredient and protein family
3. Grain/starch family
4. Cooking method
5. Texture
6. Dominant flavor
7. Spice intensity
8. Richness/oiliness
9. Serving temperature
10. Cook time and active time
11. Skill and equipment
12. Cost band
13. Batchability and leftover stability
14. Regional/cuisine affinity
15. Familiarity/popularity
16. Season/weather affinity
17. Festival/occasion affinity
18. Nutrition bands
19. Kid/elder format suitability with evidence
20. Accompaniment and composition role

Each dimension stores value, confidence, provenance, derivation method, reviewer, and version. A dish vector is rebuilt on publish; user taste vectors are updated from interactions. Safety attributes stay outside the similarity vector so a close match can never override eligibility.

51. Food knowledge graph

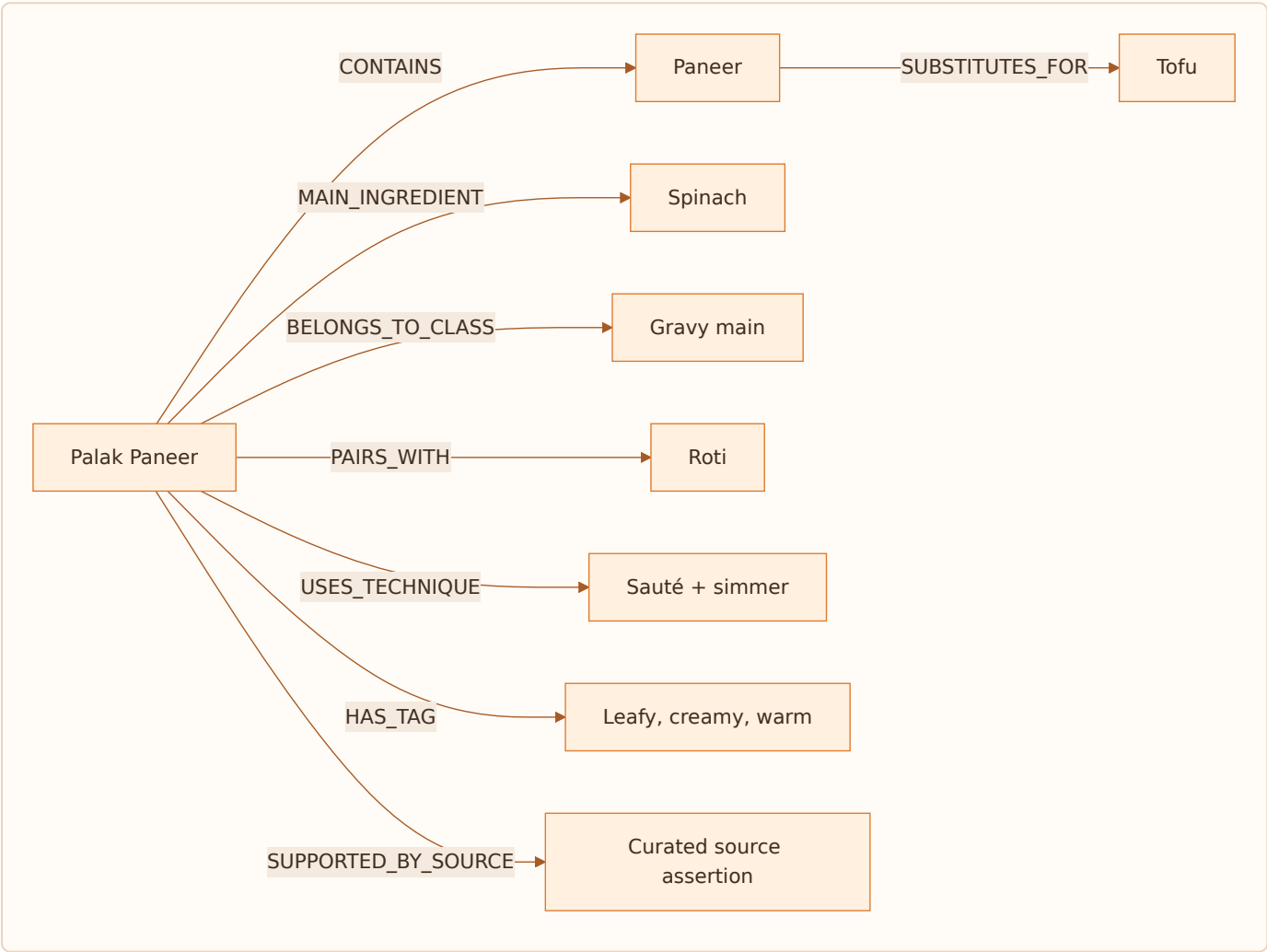
Node types

Dish, DishVariant, Ingredient, IngredientForm, MealClass, MealRole, Cuisine, Region, Technique, GenomeTag, Nutrient, Season, Festival, Equipment, Recipe, Source.

Edge types

CONTAINS, MAIN_INGREDIENT, VARIANT_OF, ALIAS_OF, BELONGS_TO_CLASS, ORIGINATES_IN, POPULAR_IN, USES_TECHNIQUE, HAS_TAG, PAIRS_WITH, SUBSTITUTES_FOR, SUITABLE_IN, SEASONAL_IN, ASSOCIATED_WITH, SUPPORTED_BY_SOURCE.

Every edge includes direction, scope/locale, confidence, provenance, effective dates, and review state.



The initial PostgreSQL implementation can store relational edges and materialize bounded adjacency; a dedicated graph database is justified only when traversal workload and operations demonstrate need.

52. Nutrition intelligence

Nutrition is represented as ranges per serving with source and uncertainty. Recipe variation makes false precision dangerous.

Data model

```
nutrient_assertion(dish_or_recipe, nutrient_code, min_value, expected_value, max_value, unit,
serving_basis, source_id, method, confidence, version).
```

Ingredient-level composition can estimate a recipe; cooking-yield and retention factors adjust values where supported. User-visible values show a range or “estimated” label unless laboratory or trusted packaged-food data supports precision.

Product use

- v1: broad balance labels and transparent data confidence.
- v2: configurable non-clinical goals such as protein-forward or lighter meals.
- v3: weekly nutrient range optimization with serving awareness.
- clinical conditions: only after qualified evidence review, clear scope, escalation rules, and legal/product approval.

WHO emphasizes diverse diets and notes that individual needs and local customs matter; Foofoo therefore avoids a single universal “health score.” Sources: [WHO healthy diet](#), [WHO India nutrition](#).

53. Regional intelligence

Regional preference is a blended, learnable signal:

$\text{RegionAffinity} = a \cdot \text{HomeIdentity} + b \cdot \text{CurrentLocale} + c \cdot \text{HouseholdHistory} + d \cdot \text{SeasonalAvailability}$, with $a+b+c+d=1$.

Home identity is not current GPS. A Bhopal-origin household in Pune may want MP comfort food, local Maharashtrian options, and global dishes at different rates. The blend begins from onboarding confidence and adapts from evidence.

Regional data

- canonical regions and culinary zones;
- dish-region affinity with type (`origin`, `popular`, `available`, `festival`);
- language labels and aliases;
- region-specific serving roles and meal composition;
- ingredient availability and substitution;
- season/festival calendars with locale scope;
- confidence and source for every assertion.

Avoid stereotyping: region is a prior, never a restriction unless the user chooses one. Users can increase “food from home” or “local discovery.” Diaspora locale packs overlay ingredient availability without erasing identity.

54. Household and life-stage intelligence

Household attributes are conditions and overlays, not fixed persona IDs. Relevant dimensions include age band, texture capability, meal role, cook assignment, schedule, diet, allergy, fasting, and temporary context.

Base plus add-on pattern

Household need	Base	Add-on/adaptation
Toddler	Mild shared khichdi	texture/portion adaptation
Diabetic elder	Shared veg main	portion/accompaniment alternative, only evidence-backed
Fitness member	Shared dal/veg	additional protein component
Fasting member	Normal household base	separate fasting-safe plate
Mixed veg/non-veg	Vegetarian shared base	optional meat/egg protein

The system optimizes additional burden and ingredient overlap. Temporary overlays have effective dates; they expire unless renewed. Sensitive data access is minimized, and explanations reveal only what the viewer is permitted to know.

55. Recipe, substitution, and main-ingredient intelligence

A dish is an identity; a recipe is an executable formulation. Multiple recipes may implement one dish with different region, time, equipment, serving, and nutrition.

Main ingredient heuristic

Flag an ingredient as main when removing it would change the dish's identity, it dominates recognized name/volume/protein role, or it defines the culinary structure. Multi-main dishes are allowed. Store confidence and evidence; do not force one main ingredient for convenience.

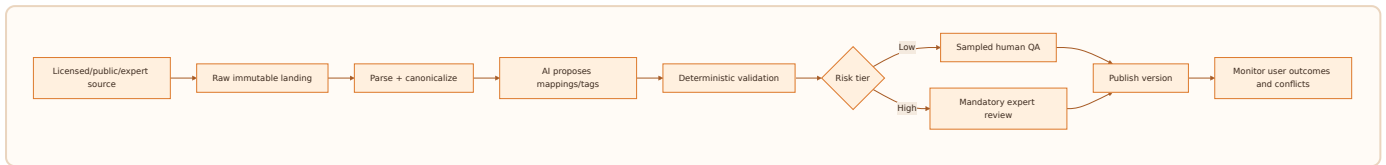
Substitution edge

A SUBSTITUTES_FOR B includes function, recipe context, ratio, preparation adjustment, diet/allergen consequences, flavor/texture delta, locale availability, confidence, and source. A global "paneer ↔ tofu" assertion without context is too coarse.

Recipe selection

Choose recipe variant after the dish using cook skill, time, equipment, serving count, availability, and locale. Re-run safety on exact ingredients and substitutions. Recipe generation by AI remains draft until ingredient identity, quantities, cooking safety, and constraint checks pass.

56. Food data acquisition and AI seeding

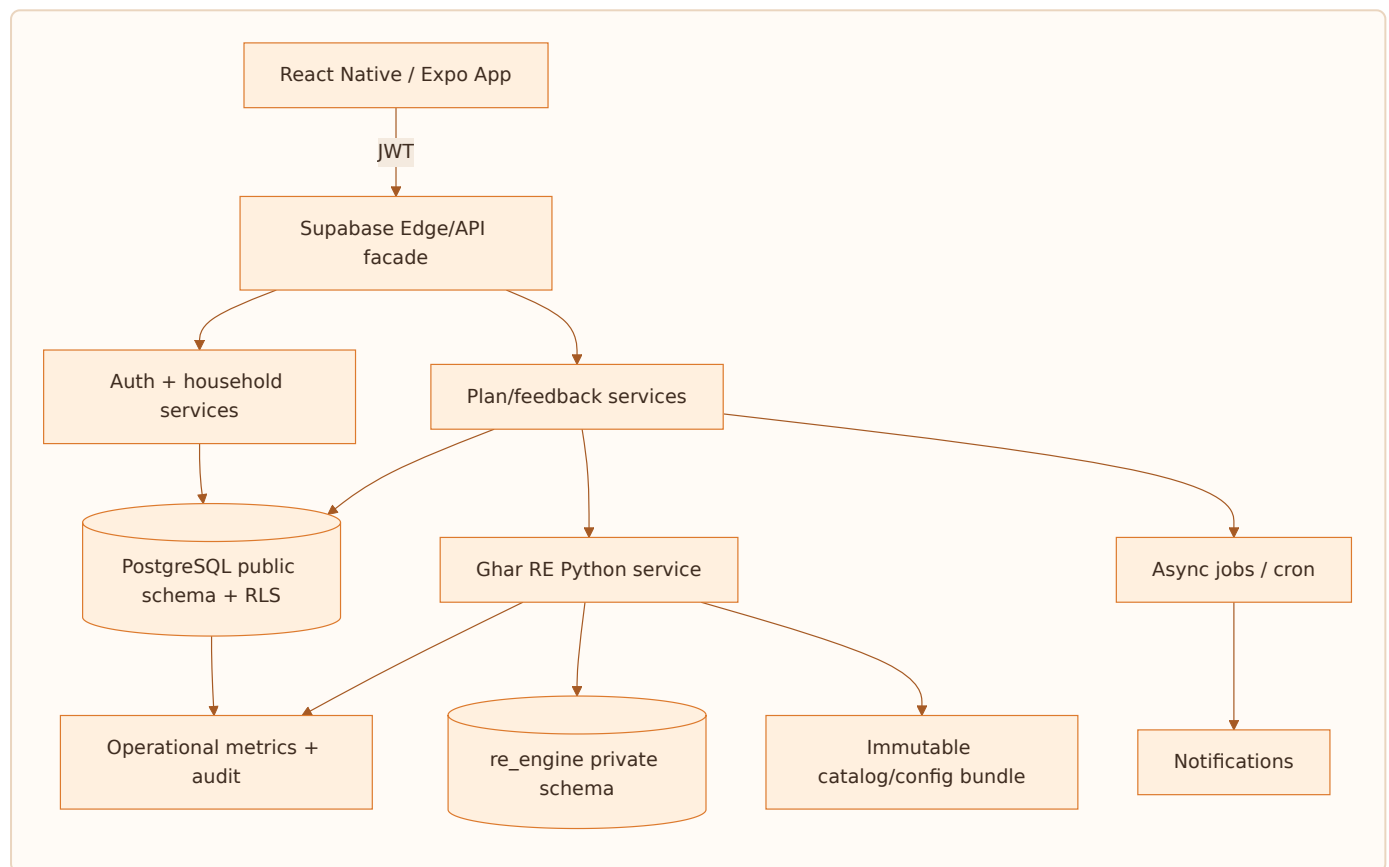


AI may create aliases, draft descriptions, candidate tags, image briefs, and low-risk relationship proposals. It may not autonomously publish allergen status, religious suitability, clinical claims, or nutrient values without trusted evidence. Each AI assertion records model, prompt/template, inputs, timestamp, confidence, validator, and reviewer state.

Raw source data remains immutable and license-tagged. Canonicalization is reproducible. Conflicts create a review task rather than silent last-write-wins behavior.

Part IV — Engineering & Architecture Bible

57. Platform architecture



The mobile client is untrusted. It talks through Supabase authentication and narrow Edge Functions. Public user-owned tables use RLS. Private recommendation data is service-role-only. The RE service is stateless per request and loads immutable catalog/config versions at startup; this enables horizontal scale and deterministic traces.

Current repository stack: React Native/Expo, Supabase Auth/Postgres/Edge Functions, Python recommendation service intended for a container platform, OneSignal notifications, and scheduled jobs. The architecture keeps direct table access for simple user-owned reads and uses custom endpoints for cross-domain invariants.

58. Service boundaries

Service/domain	Owns	Does not own
Identity/Auth	session and auth user	meal preferences
Household	profile, members, roles, consent-linked facts	ranking weights
Catalog	dishes, ingredients, recipes, ontology publish	user outcomes
Planning	week plans, slots, locks, lifecycle	learned model training
Recommendation	candidate/scoring/ranking/trace	auth source of truth
Feedback	idempotent interaction ingest	synchronous model retrain
Context	weather, season, festival snapshot	permanent user taste
Notification	eligibility, scheduling, delivery logs	plan generation truth
Experiment	assignment and exposure	arbitrary UI analytics
Privacy	export/delete workflows	business analytics definitions

Boundaries are transactional where invariants demand it. A lock and selected-slot update occur atomically. Interaction ingest is append-only and asynchronous feature updates are retryable. The recommendation response is persisted before exposure logging so the exact slate can be reconstructed.

59. API surface overview

Method	Endpoint	Purpose	Idempotency
POST	/v1/consent	Append consent decision	required
POST	/v1/onboarding	Persist/derive household onboarding	required
POST	/v1/households	Create household	required
POST	/v1/households/{id}/members	Add/update member	required
POST	/v1/recommendations	Generate/read recommendation slate	request ID
GET	/v1/plan/{household_id}/{week}	Read persisted plan	n/a
POST	/v1/plan/refresh	Refresh unlocked scope	required
POST	/v1/plan/slots/{id}/lock	Lock/unlock	required
POST	/v1/events	Append interaction/outcome	event key required
GET	/v1/search	Constraint-safe dish search	n/a
GET	/v1/user/export	Request/read export	request ID
POST	/v1/user/delete	Begin deletion	required
GET	/v1/health	Liveness/readiness/version	n/a

All private endpoints validate JWT, household membership, role permission, JSON schema, size, and rate. Error bodies expose stable codes, correlation ID, retryability, and safe user copy—not stack traces.

60. Recommendation API specification

Request

```
{
  "request_id": "uuid",
  "household_id": "uuid",
  "plan_date": "2026-08-05",
  "meal_slots": ["breakfast", "lunch", "dinner"],
  "refresh_scope": "unlocked_only",
  "client_context": {"timezone": "Asia/Kolkata", "locale": "en-IN"}
}
```

Response

```
{
  "request_id": "uuid",
  "plan_id": "uuid",
  "model_version": "re-v2.3.0",
  "catalog_version": "food-2026-08-05.1",
  "slots": [{
    "slot_id": "uuid", "meal_slot": "dinner", "selected_dish_id": "uuid",
    "alternatives": [{"dish_id": "uuid", "rank": 1, "reason_tags": ["quick", "familiar",
"variety"]}],
    "locked": false, "addon_slots": []
  }],
  "degraded": false,
  "correlation_id": "uuid"
}
```

Responses never expose raw private features or member conditions. 409 signals optimistic-lock conflict; 422 signals no safe coverage with an actionable code; 429 includes retry-after; 503 identifies cached-fallback eligibility.

61. Event tracking specification

Canonical envelope

event_id, idempotency_key, event_name, occurred_at, received_at, user_id, household_id, session_id, request_id, slate_id, item_id, rank, surface, schema_version, model_version, experiment_assignments, properties, consent_basis.

Core event taxonomy

Domain	Events
Lifecycle	app_opened, onboarding_started/completed, plan_viewed
Exposure	slate_exposed, dish_impression, explanation_viewed
Preference	dish_accepted, dish_swiped_past, dish_not_today, dish_never, never_restored
Planning	slot_locked/unlocked, slot_refreshed, dish_added_to_date, plan_shared
Outcome	recipe_started/completed, dish_cooked, dish_ordered, dish_replaced, dish_rated
Household	member_added/updated/removed, invite_sent/accepted
Reliability	fallback_served, offline_queue_flushed, safety_gate_blocked
Privacy	consent_changed, export_requested/completed, deletion_requested/completed

Events are append-only. Server timestamps preserve receipt order; device timestamps preserve user order. Duplicate keys return success without double learning.

62. Database design principles and provenance classes

Naming uses lowercase `snake_case` ; plural table names; UUID primary keys; `*_id` foreign keys; UTC `timestampz` ; ISO/canonical codes; explicit status enums/checks; `created_at` , `updated_at` , optional `deleted_at` ; JSONB only for truly variable payloads; normalized master data; append-only facts; RLS on personal tables; private schemas for engine internals.

Required provenance classes

Code	Requested category	Definition	Examples
APP	Strictly app-generated	Operational state created by product workflows	plans, slots, locks, consent records
EXT	Master strictly from external seeding	Canonical reference imported from licensed/trusted sources	regions, ingredients, nutrient masters
AI	AI-created and seeded	Proposed/enriched content with model lineage and review state	draft tags, descriptions, aliases
USE	User-usage generated	Behavioral facts emitted through use	impressions, swipes, cooks, taste vectors
MIX	App + AI + initial seed	Hybrid entity with seeded base, AI enrichment, app operations, and/or usage updates	dishes, recipes, popularity features

Every mutable row carries `data_origin` , `source_id` , `source_version` , `confidence` , `review_status` , and `lineage_metadata` where applicable. Column-level provenance below overrides table-level default.

63. Expected DB schema — identity, household, consent

Table [class]	Expected columns
public.profiles [APP]	id PK/FK auth.users, primary_cook_name, home_region_id FK, current_city_id FK, migration_duration_band, diet_type_code, religious_preference_code, cook_capability_code, onboarding_completed, notification_time, locale, timezone, created_at, updated_at, deleted_at
public.households [APP]	id PK, name, household_type_code, owner_user_id, default_locale, timezone, status, timestamps
public.household_members [APP]	id PK, household_id FK, user_id FK nullable, display_name, age_band_code, member_role_code, diet_type_code, religious_preference_code, allergen_mask, conditions[], is_active, effective_from/to, timestamps
public.household_invites [APP]	id, household_id, token_hash, invited_role, expires_at, accepted_at, revoked_at, timestamps
public.onboarding_sessions [APP]	id, profile_id, household_id, screen_id, question_key, answer_value jsonb, skipped, answered_at, schema_version
public.household_answers [APP]	id, household_id, answer_key, answer_value, source, confidence, effective_from/to, timestamps
public.consent_records [APP]	id, profile_id, consent_type, granted, policy_version, ip_hash, granted_at
public.privacy_requests [APP]	id, profile_id, request_type, status, requested_at, completed_at, artifact_uri, error_code

Sensitive columns use restricted access; analytics receives pseudonymous IDs only.

64. Expected DB schema — food master and ontology

Table [class]	Expected columns
public.dishes [MIX]	id, canonical_name, parent_dish_id, description, meal_occasions[], cook_time_minutes, active_time_minutes, difficulty_code, diet_type_code*, is_jain*, allergen_mask*, genome_vector*, popularity_score*, photo_url, data_origin, source_id, confidence, review_status, timestamps
public.dish_names [MIX]	id, dish_id, locale, name, name_type, is_preferred, source_id, confidence, review_status
public.ingredients [EXT]	id, canonical_name, ingredient_category_code, is_veg, is_vegan, is_jain_excluded, allergen_mask, seasonal_peaks[], source_id, source_version, review_status
public.ingredient_names [MIX]	id, ingredient_id, locale, name, name_type, provenance columns
public.dish_ingredients [MIX]	dish_id, ingredient_id, quantity, unit_code, preparation, is_optional, is_main, main_confidence, provenance columns
public.tags [EXT]	id, tag_code, tag_family, definition, vector_position, status, provenance columns
public.dish_tags [MIX]	dish_id, tag_id, weight, confidence, derivation_method, source_id, review_status, timestamps
public.cuisines [EXT]	id, cuisine_code, name, parent_id, region_id, provenance columns
public.regions [EXT]	id, region_code, name, region_type, parent_id, country_code, provenance columns
public.meal_classes [EXT]	id, class_code, name, meal_slots[], planning_role, definition, status

* denotes derived-stored: written only by controlled jobs/triggers, never client input.

65. Expected DB schema — recipes, nutrition, graph

Table class	Expected columns
public.recipes [MIX]	id, dish_id, locale, title, servings, total_time_minutes, active_time_minutes, difficulty_code, equipment_codes[], instructions_status, provenance/review/version columns
public.recipe_steps [MIX]	id, recipe_id, step_number, instruction, duration_seconds, equipment_code, media_url, provenance columns
public.recipe_ingredients [MIX]	recipe_id, ingredient_id, quantity, unit_code, preparation, is_optional, substitution_group_id
food.nutrients [EXT]	id, nutrient_code, name, unit, upper_lower_semantics, source/version
food.nutrient_assertions [MIX]	id, subject_type, subject_id, nutrient_id, min_value, expected_value, max_value, serving_basis, method, confidence, source/review/version
food.ontology_nodes [MIX]	id, node_type, canonical_entity_id, label, locale, status, provenance
food.ontology_edges [MIX]	id, subject_node_id, predicate_code, object_node_id, scope_region_id, weight, confidence, effective_from/to, provenance/review
food.substitutions [MIX]	id, from_ingredient_id, to_ingredient_id, function_code, recipe_context, ratio, adjustment_text, constraint_delta, confidence/provenance
food.dish_combos [MIX]	id, name, meal_slot, region_id, status, provenance
food.dish_combo_items [MIX]	combo_id, dish_id, component_role, is_required, sequence, portion_ratio

The `food` schema is proposed for the richer v3 ontology; the current repository uses `public` plus private RE tables and a bounded relational graph.

66. Expected DB schema — plans and interactions

Table <code>class</code>	Expected columns
<code>public.week_plans</code> [APP]	<code>id</code> , <code>household_id</code> , <code>week_start_date</code> , <code>status</code> , <code>generation_request_id</code> , <code>model_version</code> , <code>catalog_version</code> , <code>experiment_snapshot</code> , <code>timestamps</code> , <code>version</code>
<code>public.plan_slots</code> [APP]	<code>id</code> , <code>week_plan_id</code> , <code>plan_date</code> , <code>meal_slot_code</code> , <code>selected_dish_id</code> , <code>selected_combo_id</code> , <code>slate_id</code> , <code>status</code> , <code>is_locked</code> , <code>locked_by</code> , <code>locked_at</code> , <code>context_snapshot_id</code> , <code>version</code> , <code>timestamps</code>
<code>public.addon_slots</code> [APP]	<code>id</code> , <code>plan_slot_id</code> , <code>household_member_id</code> , <code>addon_class_id</code> , <code>selected_dish_id</code> , <code>reason_code</code> , <code>status</code> , <code>timestamps</code>
<code>public.slates</code> [APP]	<code>id</code> , <code>request_id</code> , <code>plan_slot_id</code> , <code>surface</code> , <code>model_version</code> , <code>config_version</code> , <code>created_at</code> , <code>expires_at</code>
<code>public.slate_items</code> [APP]	<code>slate_id</code> , <code>dish_id</code> , <code>rank</code> , <code>point_score</code> , <code>rerank_score</code> , <code>generator_codes[]</code> , <code>reason_tags[]</code> , <code>selection_propensity</code>
<code>public.interaction_events</code> [USE]	<code>id</code> , <code>idempotency_key</code> , <code>event_name</code> , <code>user_id</code> , <code>household_id</code> , <code>slate_id</code> , <code>dish_id</code> , <code>rank</code> , <code>surface</code> , <code>occurred_at</code> , <code>received_at</code> , <code>schema_version</code> , <code>properties</code> , <code>consent_basis</code>
<code>public.outcome_events</code> [USE]	<code>id</code> , <code>plan_slot_id</code> , <code>dish_id</code> , <code>outcome_type</code> , <code>value</code> , <code>occurred_at</code> , <code>source</code> , <code>confidence</code>
<code>public.suggestion_logs</code> [USE]	<code>id</code> , <code>request_id</code> , <code>household_id</code> , <code>slot</code> , <code>candidate_counts</code> , <code>selected_ids</code> , <code>model/config/catalog versions</code> , <code>latency_ms</code> , <code>degraded</code> , <code>created_at</code>
<code>public.context_log</code> [APP]	<code>id</code> , <code>request_id</code> , <code>household_id</code> , <code>weather_code</code> , <code>season_code</code> , <code>festival_ids[]</code> , <code>day_type</code> , <code>time_budget</code> , <code>snapshot_hash</code> , <code>created_at</code>

Events and logs are time-partitioned and append-only, with privacy retention policies.

67. Expected DB schema — RE state and configuration

Table <code>[class]</code>	Expected columns
<code>re_engine.re_cohorts</code> [EXT]	<code>id</code> , <code>cohort_code</code> , dimension codes, region/city tiers, status, seed version
<code>re_engine.re_personas</code> [EXT]	<code>id</code> , <code>persona_code</code> , <code>name</code> , <code>main_cohort_id</code> , <code>subcohort_id</code> , description, seed version
<code>re_engine.re_routing_rules</code> [EXT]	<code>id</code> , <code>rule_code</code> , conditions jsonb, target code, priority, effective dates, seed version
<code>re_engine.re_class_dish_options</code> [MIX]	<code>meal_class_id</code> , <code>dish_id</code> , <code>base_weight</code> , <code>region_id</code> , <code>eligibility_status</code> , provenance/review
<code>re_engine.re_weekly_class_plans</code> [EXT]	<code>id</code> , <code>cohort_id</code> , <code>day_index</code> , <code>meal_slot</code> , <code>class_id</code> , <code>weight</code> , seed version
<code>re_engine.re_cohort_class_priors</code> [MIX]	<code>cohort_id</code> , <code>class_id</code> , <code>meal_slot</code> , <code>prior_score</code> , <code>sample_size</code> , <code>calibrated_at</code> , source
<code>re_engine.user_re_state</code> [USE]	<code>household_id</code> , <code>confidence</code> , <code>interaction_count</code> , <code>cold_start_state</code> , <code>model_version</code> , <code>last_updated_at</code>
<code>re_engine.user_taste_vectors</code> [USE]	<code>household_id</code> , <code>vector_type</code> , <code>vector real[]</code> , <code>evidence_count</code> , <code>updated_at</code> , <code>feature_version</code>
<code>re_engine.member_taste_vectors</code> [USE]	<code>member_id</code> , vector/evidence/version columns
<code>re_engine.never_list</code> [USE]	<code>household_id</code> , <code>member_id</code> nullable, <code>entity_type</code> , <code>entity_id</code> , <code>source_event_id</code> , <code>created_at</code> , <code>restored_at</code>
<code>re_engine.not_today_suppression</code> [USE]	<code>household_id</code> , <code>dish_id</code> , <code>penalty</code> , <code>starts_at</code> , <code>expires_at</code> , <code>source_event_id</code>
<code>re_engine.variety_window_state</code> [USE]	<code>household_id</code> , <code>dimension_code</code> , <code>entity_id</code> , <code>last_seen_at</code> , <code>count_in_window</code> , <code>updated_at</code>
<code>re_engine.bandit_state</code> [USE]	<code>policy_id</code> , <code>subject_type/id</code> , <code>context_bucket</code> , posterior parameters, impressions, rewards, <code>updated_at</code>

Private RE tables are reachable only through service roles and audited interfaces.

68. Expected DB schema — config, operations, lineage

Table [class]	Expected columns
re_engine.scoring_config [EXT]	config_version, named parameters, effective_from, status, checksum
re_engine.weight_ladder_config [EXT]	config_version, confidence_tier, signal weights, bounds
re_engine.event_weights [EXT]	config_version, event_name, weight, half_life_days, context rules
re_engine.variety_rules [EXT]	rule_code, dimension, window, cap, override conditions, version
re_engine.context_multipliers [EXT]	context code, genome tag, multiplier, confidence, version
re_engine.engine_versions [APP]	version, artifact_uri, checksum, status, activated_at, rollback_of
ml.feature_definitions [APP]	feature_name, type, owner, expression/version, null policy, online/offline source, status
ml.feature_values [USE]	entity_type/id, feature_name, value, as_of, feature_version
ml.model_registry [APP]	model/version, training dataset, metrics, artifact/checksum, stage, approver, timestamps
ml.experiment_assignments [APP]	experiment_id, household_id, variant, assigned_at, assignment_version
ops.safety_gate_log [APP]	request/slate/dish, gate, code, evidence, model/catalog version, created_at
ops.coverage_gap_log [APP]	request, region/class/constraints hash, counts, fallback, created_at
ops.etl_job_runs [APP]	job, run, input/output versions, counts, status, error, timestamps
ops.data_sources [EXT]	source_id, owner, license, URI, retrieval date, checksum, permitted uses
ops.ai_generation_runs [AI]	run_id, model, prompt version, input source IDs, parameters, output artifact, validator result, reviewer, timestamps
ops.audit_log [APP]	actor, action, resource, before/after hash, correlation, occurred_at

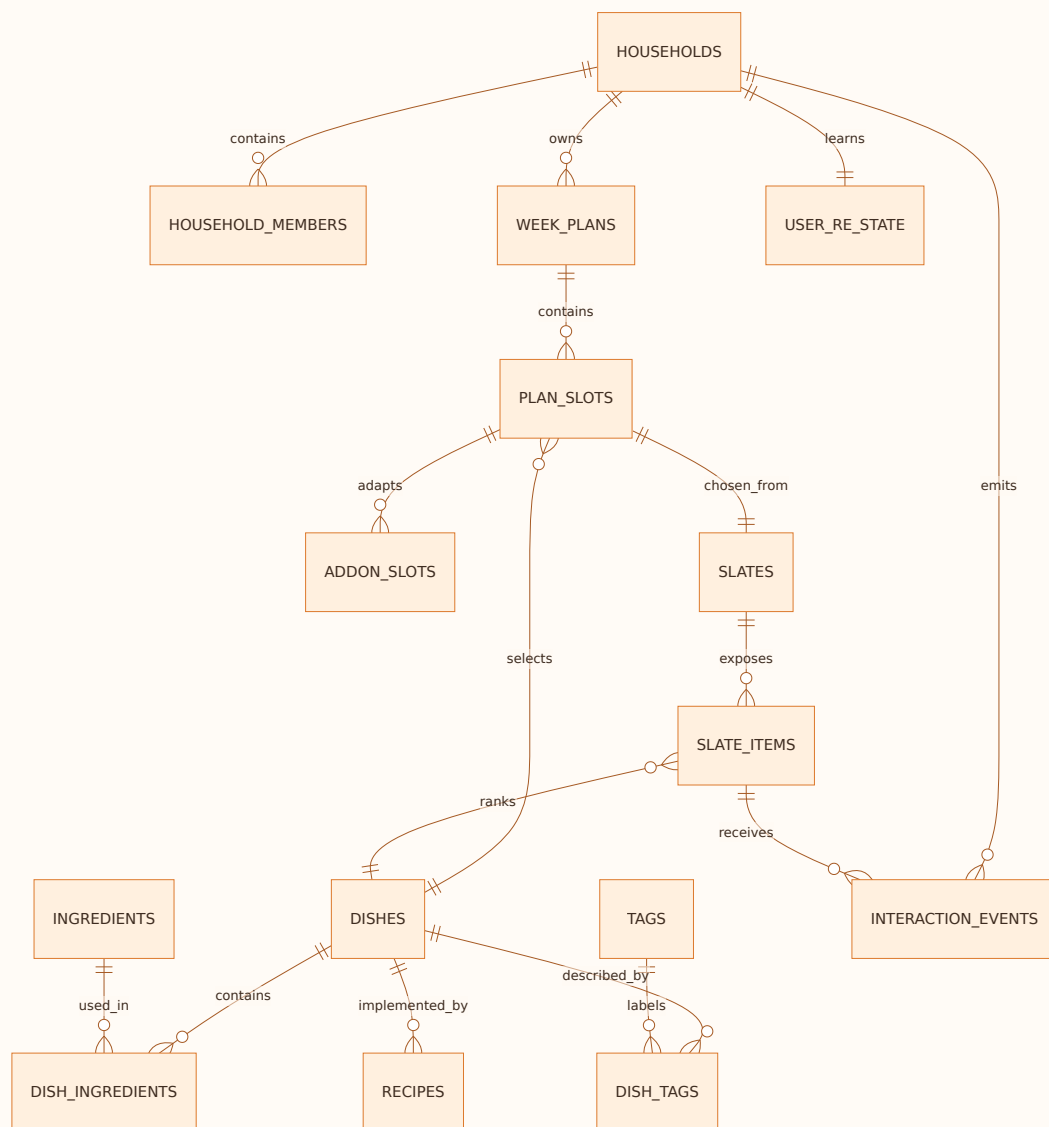
69. Column-level provenance rules

Hybrid tables need explicit ownership:

Field family	Permitted origin	Write authority
Canonical external identity/name	EXT	ingestion publisher only
AI description/alias/tag proposal	AI	generation pipeline into draft
Reviewed description/tag	MIX	content publisher after validation
diet_type , is_jain , allergen_mask	MIX-derived from EXT/MIX ingredients	trigger/derivation job only
genome_vector	MIX-derived	versioned feature build only
popularity_score , acceptance rates	USE-derived	scheduled feature job only
Plan/slot/lock status	APP	planning service transaction
Swipes/cooks/orders	USE	event ingest append only
Taste vectors/bandit posterior	USE-derived	learning workers only
Model/config active status	APP	deployment control plane

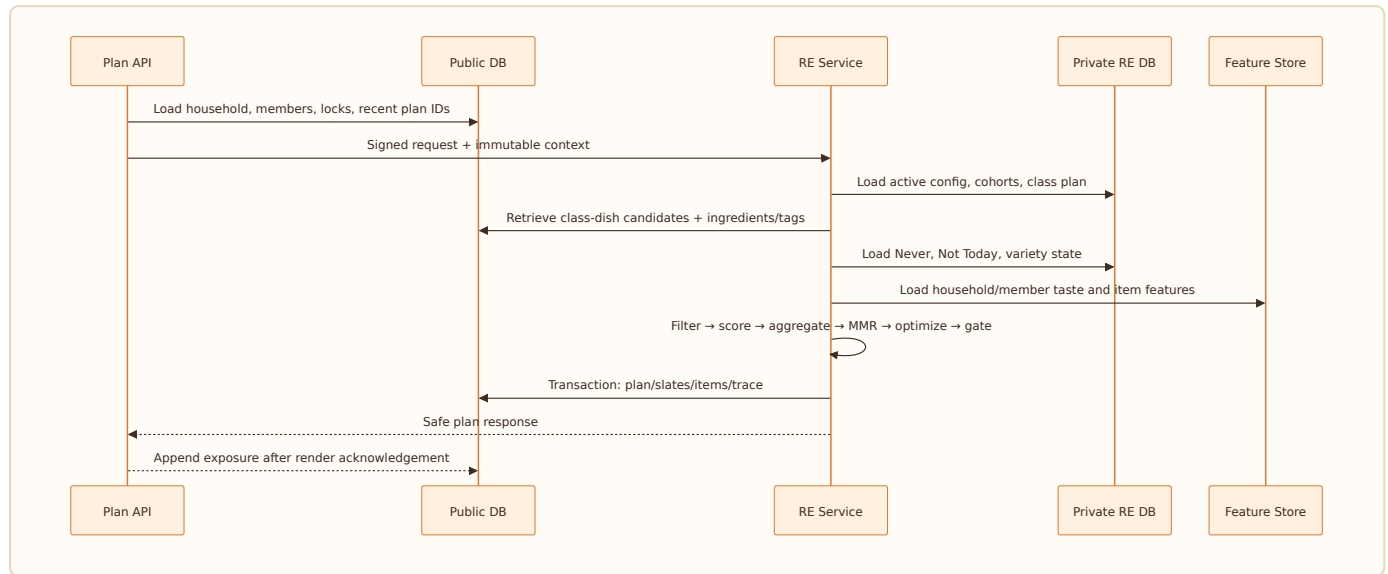
Database roles reflect these boundaries: `client_authenticated` , `edge_app` , `catalog_ingest` , `catalog_publisher` , `re_runtime` , `feature_writer` , `privacy_worker` , and `read_analytics` . Supabase platform roles map to these logical authorities through functions and grants. No general application role may manually update derived safety columns.

70. Entity relationships



Cardinality and cascade policy: user-owned plans and events delete through the privacy workflow; master dishes never cascade-delete into historical events—instead IDs remain and content is deactivated. Junction rows cascade with their master. Append-only trace tables retain referenced identifiers or immutable snapshots to preserve replay until retention expiry.

71. How data is pulled for one recommendation



Query plan

1. Authorize `auth.uid()` against household membership.
2. Read one household snapshot with members and constraints.
3. Read locked slots and recent-window features by indexed household/date keys.
4. Fetch class IDs for requested slots and cohort.
5. Batch fetch class candidates; join canonical dish, ingredient safety, tags, regional affinity.
6. Anti-join Never; apply temporary suppression as a feature/penalty.
7. Batch feature lookup; never issue one query per candidate.
8. Persist decision artifacts atomically, then return.

72. Indexing, partitioning, consistency

Required indexes

- household ownership: `(household_id, user_id)` and active partial indexes;
- plan read: `(household_id, week_start_date)` unique, slots `(week_plan_id, plan_date, meal_slot)`;
- events: `(household_id, occurred_at desc)`, `(slate_id, dish_id)`, unique idempotency key;
- content joins: `(meal_class_id, dish_id)`, `(dish_id, ingredient_id)`, `(dish_id, tag_id)`;
- suppressions: active partial `(household_id, dish_id)` where `restored_at` is null;
- search: GIN `tsvector` on names/aliases; vector index only after measured need;
- trace: `(request_id)`, `(household_id, created_at desc)`.

Interactions, suggestion logs, traces, and feature history partition monthly by `occurred_at/created_at`. Partition creation is automated and monitored. Personal rows use soft deletion only during the bounded erasure workflow; purge removes partitions' matching records using controlled jobs.

Optimistic concurrency uses a `version` integer or updated timestamp. Plan locks use row-level transactions. Catalog publishes are immutable snapshots selected by version; an in-flight request never mixes catalog versions.

73. Data quality and master-data operations

Publish gates

Entity	Minimum gate
Dish	canonical name, class, occasion, active recipe or display disclaimer, source
Ingredient	canonical identity, diet/allergen/Jain fields, source
Dish-ingredient	complete safety-relevant ingredient set
Tag	valid vocabulary and vector position
Class mapping	safe candidate count by priority cohort
Recipe	ordered steps, quantities, servings, exact safety pass
Nutrition	source, serving basis, unit, range/confidence
Graph edge	valid nodes, predicate schema, provenance, review state

Quality dashboards show completeness, conflicts, orphan relations, duplicate aliases, unsafe derivation differences, class coverage, region coverage, stale assertions, and AI proposal acceptance rates. The content operations queue is prioritized by real coverage gaps and high-impression uncertainty, not by arbitrary catalog size.

74. Security and privacy architecture

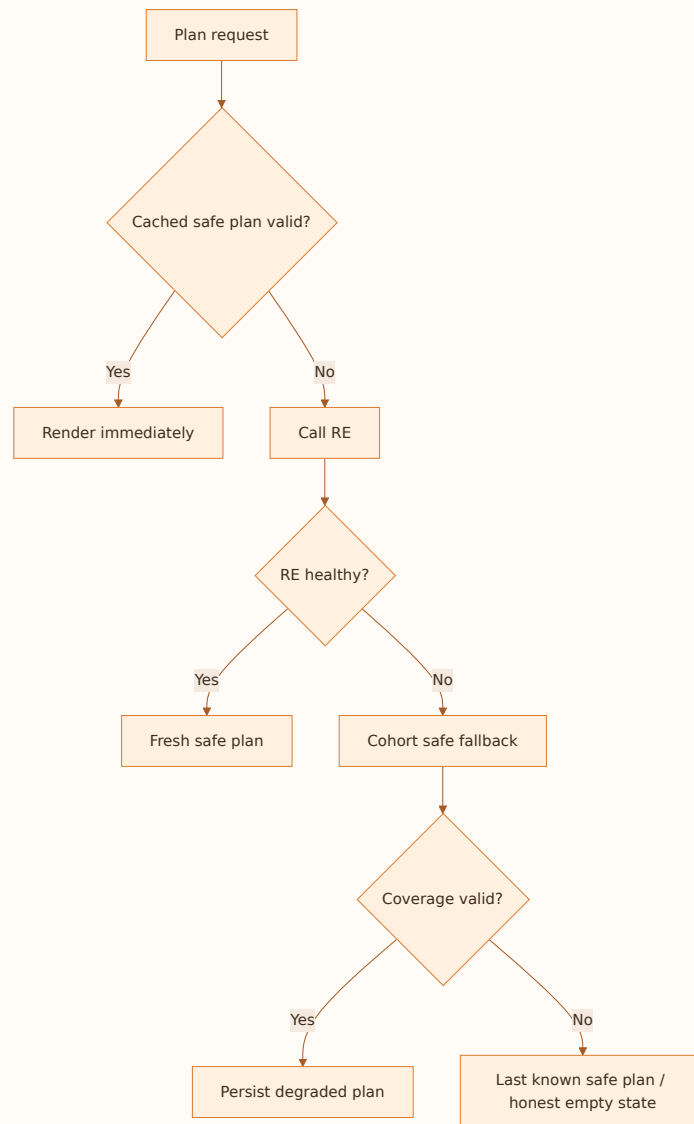
Trust boundaries are mobile, public API, private services, databases, external providers, and analytics. Controls include JWT validation, row-level security, least-privilege service roles, schema isolation, encrypted transport/storage, secret management, rate limiting, validation, dependency scanning, and auditable admin actions.

Threat examples

Threat	Control
Read another household plan	RLS + explicit ownership check
Forge feedback for another user	server derives user from JWT; idempotent event validation
Prompt injection through AI food content	AI output treated as untrusted data; schema validation and review
Infer a member's health condition	privacy-preserving constraint flags; explanation redaction
Leak service key in mobile	service keys only in server secret store; CI scanning
Poison ranking via bots	behavioral rate/abuse detection, robust aggregation, quarantine
Unsafe catalog edit	restricted publisher role, derivation triggers, safety regression

Data minimization maps every collected field to a product purpose. Export and deletion span public data, private RE state, events, derived features, notification identity, and cached artifacts.

75. Reliability, failure modes, and fallback



Failure policy prefers a transparent limited experience over a fabricated plan. Timeouts are bounded. Retries use exponential backoff with jitter only for idempotent operations. Circuit breakers stop cascade failure. Dead letters contain identifiers and error codes, not unnecessary PII.

Backups have tested restore procedures and explicit RPO/RTO. Schema migrations are forward-compatible, transactional where possible, and use expand/migrate/contract. The mobile app tolerates at least one prior API version during rollout.

76. Observability and SLOs

Golden signals

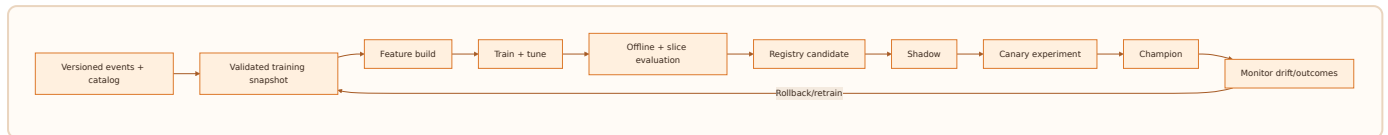
- latency by endpoint/stage/model/cohort;
- traffic and active households;
- errors, fallbacks, safety gates, empty candidate pools;
- saturation of DB connections, CPU, memory, queue age;
- product outcomes: WSMD, acceptance, completion;
- data health: feature/catalog freshness and event lag.

SLO examples

SLI	Objective
Plan-read availability	99.9% monthly
Fresh recommendation success	99.5% excluding invalid requests
Cached plan p95	<1 second end-to-end
Fresh plan p95	<3 seconds end-to-end
Feedback ingest	99.9% accepted or deduplicated
Safety gate false-negative	0 known
Event-to-feature lag	p95 <5 minutes for near-real-time features

Logs use correlation/request/slate IDs; traces span edge, RE, DB, and external context; metrics avoid high-cardinality raw user IDs. Alerts are actionable and point to runbooks, recent deploys, and rollback controls.

77. MLOps lifecycle



Training snapshots are immutable and consent-filtered. Model artifacts include code commit, environment lock, feature definitions, dataset range, metrics by slice, calibration, safety tests, owner, and checksum. Online and offline transformations share code or are parity-tested.

Shadow models receive production feature snapshots without affecting users. Canary ramp follows 1% → 5% → 25% → 50% → 100%, conditioned on SLO and product guardrails. Rollback is a pointer change. A model cannot become champion if its food catalog version is unavailable.

78. Infrastructure and deployment

Environments

Local, test, staging, and production use separate projects/secrets, equivalent schema, and sanitized test data. Production deployment is explicit and auditable. Infrastructure and DB migrations precede compatible service rollout; mobile rollout tolerates lagging clients.

Runtime scaling

- Edge/API functions scale stateless request validation and orchestration.
- RE containers scale horizontally behind health checks.
- PostgreSQL uses connection pooling, indexed hot paths, read replicas only when measurements justify.
- Immutable catalog/config bundles reduce hot-path joins and support rollback.
- Jobs handle plan pre-generation, notifications, aggregation, retention, and partition care.
- CDN serves optimized food images.

Cost controls

Track cost per recommendation and WSMD; pre-generate for likely active households; cache context; batch feature reads; keep LLM generation offline; cap abusive refresh; right-size images and logs; archive trace detail per retention policy while retaining aggregate metrics.

79. Testing strategy

Layer	Required tests
Catalog	schema, uniqueness, provenance, safety derivation, coverage
Algorithm	unit tests for every filter/score/penalty, property-based safety
Golden personas	25 persona journeys across seasons and contexts
API	contract, auth, idempotency, errors, version compatibility
Database	migrations, constraints, triggers, RLS, query plans, rollback
Mobile	component, navigation, accessibility, offline, gesture equivalence
Load	cold/warm plans, event bursts, cron overlap, connection saturation
Chaos	RE down, context provider down, stale features, partition/job failure
ML	leakage, reproducibility, calibration, slice regression, drift
Security	abuse, injection, secret scanning, dependency and access tests

Release certification includes a safety query returning zero violations, first-plan completion on a reference budget Android device, exact trace replay, and physical-device notification validation where notifications ship.

80. Delivery plan, ownership, and release gates

Workstreams

- 1. Product/UX: flows, screens, content, usability.
- 2. Food data: catalog, ontology, recipes, regional coverage.
- 3. RE/ML: ranking, evaluation, traces, experimentation.
- 4. Platform: APIs, DB, infra, security, observability.
- 5. Growth/GTM: design partners, referral, lifecycle, pricing.

Gates

Gate	Exit condition
G0 Foundation	canonical IDs, provenance, event contracts, safety invariants approved
G1 Internal alpha	25 golden personas pass; safe fallback and trace replay work
G2 Design partner	activation/WSMD signal; critical UX issues resolved
G3 City beta	catalog coverage, latency, support load, W4 retention thresholds met
G4 Public launch	physical-device, privacy, reliability, incident, store readiness complete
G5 Monetization	demonstrated habit and value; billing/cancel/support ready

Each gate has one accountable owner per workstream and a signed evidence bundle. Dates follow evidence; release scope may shrink, but safety and traceability do not.

81. Risks and mitigations

Risk	Early indicator	Mitigation
Bad first-week relevance	high swap/Never, low top-3 success	strengthen class priors and catalog coverage
Safety/content error	gate conflicts, complaints	ingredient-ground-truth filters, quarantine, incident path
Sparse event data	low exposure/outcome linkage	simplify outcome capture, preserve full slate logs
Household minority ignored	fairness-floor decline	role-aware welfare objective and weekly audit
Over-complex onboarding	completion drop by screen	progressive disclosure, skip soft questions
Notification fatigue	opt-outs, uninstall after push	value eligibility and hard frequency caps
AI hallucinated facts	validation conflicts	draft-only AI pipeline with provenance/review
Partner pressure distorts rank	organic acceptance declines	separate sponsored surface and policy
Infrastructure cost	cost/WSMD rises	caching, batching, offline LLM, pre-generation controls
Premature ML	offline lift not reproducible	class-first baseline and promotion gates
Regional stereotyping	“not for me” feedback	region as prior; adaptive blend and controls
Medical overreach	support/legal incidents	non-clinical scope, evidence tiers, qualified review

82. Open product decisions

1. Is household voting in public v1, or tested as a lightweight link after the planner loop works?
2. Which meal slots launch by city and catalog coverage?
3. Are groceries part of free habit formation or a paid conversion feature?
4. What minimum evidence tier allows condition-aware suitability labels?
5. What threshold and definition constitute “cold-start exit”?
6. Should cook assignment be per household, day, or slot?
7. How much regional exploration can occur by default?
8. What outcome counts as success when the user neither confirms nor replaces a plan?
9. Which partner actions may appear in organic cards, if any?
10. Which locale/language follows Hindi and English?

Decisions must update requirements, event schemas, golden tests, and data definitions together. An unresolved question may use a reversible experiment; it may not create ambiguous safety behavior.

83. Definition of done

This PRD is implemented when:

- the complete core lifecycle works online and in degraded mode;
- all P0 functional and non-functional requirements have automated evidence;
- all 25 persona fixtures receive culturally plausible, feasible, safe plans;
- every displayed item has a canonical identity, ingredients, class, provenance, and review status;
- every recommendation is replayable from a stored decision trace;
- event exposure and outcome linkage are complete enough for unbiased learning;
- household safety and permissions are enforced at API and database layers;
- accessibility, budget-device performance, export, deletion, and physical notification tests pass;
- operating dashboards and incident/rollback paths exist;
- product, food data, engineering, privacy, and support owners accept their runbooks;
- public claims use validated current sources and do not imply medical authority.

Done means a household can trust Foofoo with tomorrow's decision, and the team can explain, operate, improve, and—when necessary—reverse every part of that decision.

Appendix A — Source register

Primary repository inputs (all outside excluded Archive/Governance scope):

- Product: active Product Brief, Market Research, User Personas, GTM, Revenue, Product Bible.
- UX: active PRD, Information Architecture, UX Design System, Visual Design System Explorer.
- Recommendation: canonical planning model/semantics, final RE architecture review, business logic, cold-start design, Ghar RE core spine/derivation/knowledge base, RE visuals.
- Data/API: Conceptual Domain Model, Data Architecture ERD, Migration Strategy, API Contract.
- Engineering: Technical Architecture, Backend Foundation, Service/Edge Function spec, Integration/Infrastructure, Deployment Topology, Extensibility Review.
- Research: active canonicalization, mapping, gap-analysis, discovery, and pipeline packages excluding the governance evaluation package.
- Roadmap/current state: active product roadmap, RE intelligence roadmap draft, current status, open items, and launch blockers.
- User attachment: “Detailed Plan for an AI-Powered Meal Planning App.”

External current context: [TRAJ](#), [NPCJ UPI statistics](#), [WHO India healthy diet](#), [WHO healthy diet fact sheet](#).

Appendix B — Requirement-to-system traceability

Product need	UX	Service	Data	Model/evidence
Ready plan	Today/Week	Planning + RE	plans, slots, slates	class plan + ranker
Safe household meal	Onboarding/Profile	Household + Catalog + RE	members, ingredients, constraints	filters + post-gate
Alternatives	Carousel	Recommendation	slate items/events	MMR and exposure policy
Temporary rejection	Not Today action	Feedback	suppression	decay penalty
Permanent exclusion	Never confirm/list	Feedback/Profile	never list	hard anti-join
Member adaptation	Household/add-on card	Planning	add-on slots	bundle optimizer
Useful why	Explanation sheet	Trace/Explanation	decision trace	contribution selection
Learn outcomes	Cook/order/rating	Event ingest	outcome events/features	decay, rank training
Groceries	Grocery tab	Plan-to-basket	recipes/ingredients	consolidation/substitution
Privacy control	Privacy screen	Privacy worker	consent/requests/audit	consent-filtered learning

Appendix C — Glossary

Candidate: eligible item before final rank. **Class-first planning:** selecting the kind/role of meal before selecting a dish. **Cold start:** insufficient behavioral evidence for a household, member, dish, or region. **Decision trace:** immutable evidence of inputs, filters, features, scores, versions, and result. **Hard constraint:** rule that removes a candidate and cannot be traded for relevance. **Household intelligence:** representation and aggregation of member roles, constraints, and preferences. **Meal Genome:** structured, versioned representation of sensory, culinary, practical, cultural, and nutrition properties. **MMR:** diversity re-ranking that balances relevance and similarity to already selected items. **Not Today:** strong, time-decaying rejection. **Never:** explicit persistent exclusion until restored. **Slate:** stable ordered alternatives shown for a decision. **WSMD:** Weekly Successful Meal Decisions, Foofoo's north-star outcome.

End of FooFoo Comprehensive PRD and Intelligence Bibles v1.0